



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2
ระหว่างวันที่ 18 เมษายน ถึง 28 เมษายน พ.ศ. 2565

อัลกอริทึมกราฟและการแก้ปัญหาด้วยกราฟ

อ.ลือพล พิพานเมฆาภรณ์

luepol.p@sci.kmutnb.ac.th

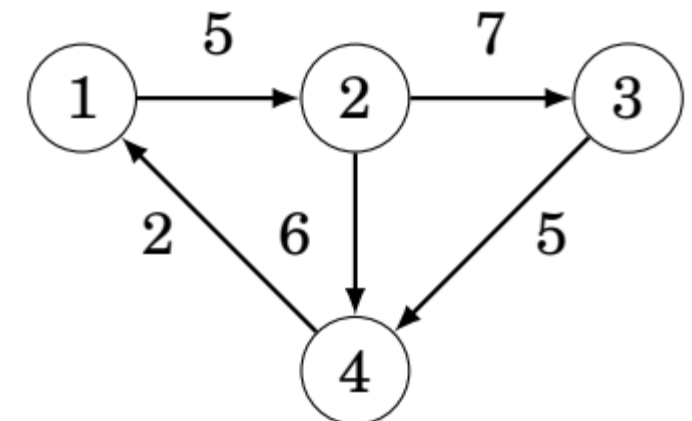
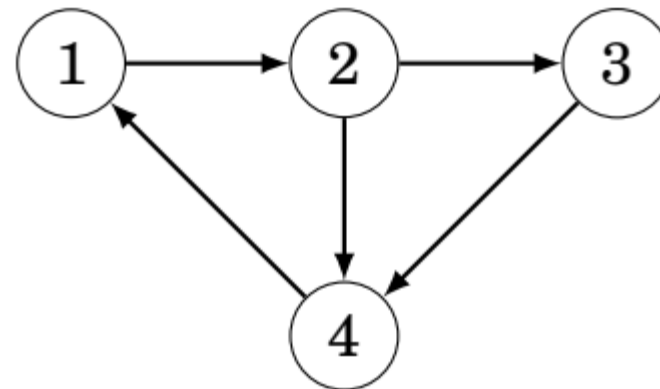
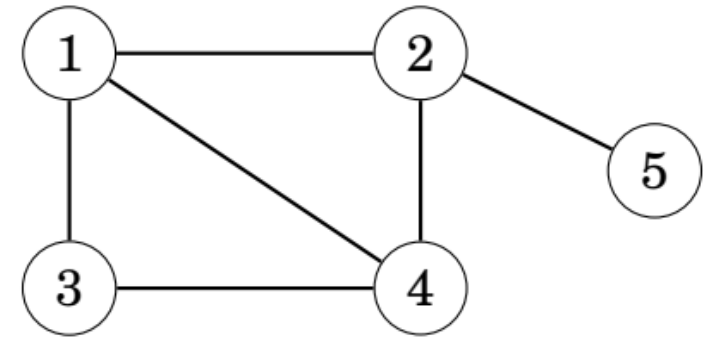
ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

- ทบทวนกราฟ
 - เมตริกซ์ประชิด (Adjacency matrix)
 - ลิสต์ประชิด (Adjacency list)
- ปัญหาการหาเส้นทาง (Path) ในกราฟ
 - อัลกอริทึม ^{warshall}Floyd และการ implement
- ปัญหาการหาเส้นทางสั้นที่สุด (Shortest path)
 - อัลกอริทึม Dijkstra และการพัฒนา
 - อัลกอริทึม ^{Floyd}Warshall และการพัฒนา
- ปัญหาการหาต้นไม้แผ่ทั่วที่เล็กที่สุด (Minimum Spanning Tree)
 - อัลกอริทึม Prim และการพัฒนา
 - อัลกอริทึม Kruskal และการพัฒนา

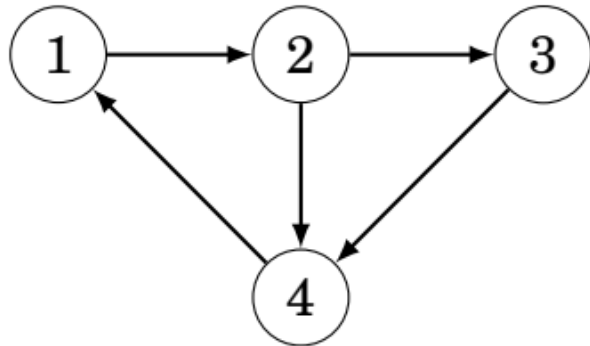
ทบทวนกราฟ

- ชนิดของกราฟ (types of graph)
 - กราฟมีทิศทางและไม่มีทิศทาง
 - กราฟมีน้ำหนักและไม่มีน้ำหนัก
- การแสดงกราฟ (graph representation)
 - วิธีเมตริกซ์ประชิด (adjacency matrix)
 - วิธีลิสต์ประชิด (adjacency list)

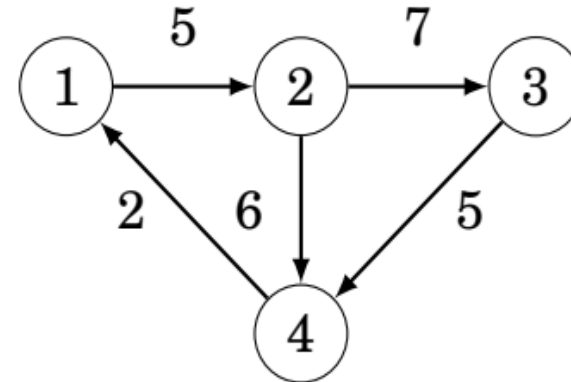


เมตริกซ์ประชิด (Adjacency matrix)

```
int adj[N][N];
```



	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	0	0	0	1
4	1	0	0	0

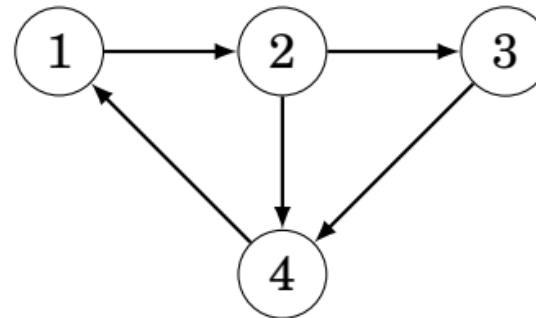


	1	2	3	4
1	0	5	0	0
2	0	0	7	6
3	0	0	0	5
4	2	0	0	0

ลิสต์ประชิด (Adjacency list)

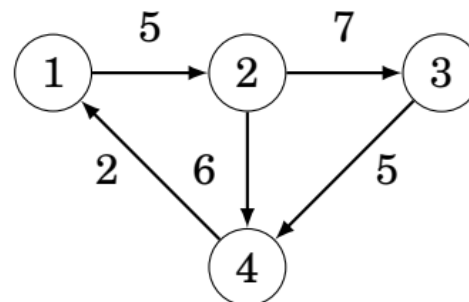
```
vector<int> adj[N];
```

```
adj[1].push_back(2);
adj[2].push_back(3);
adj[2].push_back(4);
adj[3].push_back(4);
adj[4].push_back(1);
```



```
vector<pair<int,int>> adj[N];
```

```
adj[1].push_back({2,5});
adj[2].push_back({3,7});
adj[2].push_back({4,6});
adj[3].push_back({4,5});
adj[4].push_back({1,2});
```



การเข้าถึงสมาชิกในลิสต์ประชิด

กราฟไม่มีน้ำหนัก

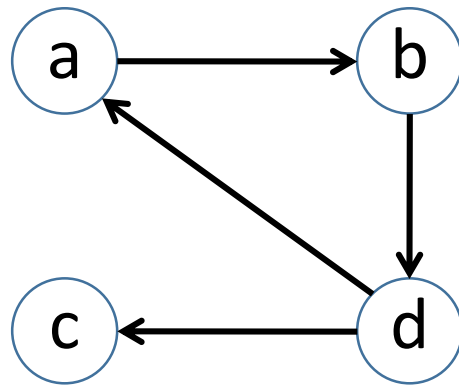
```
for (auto v : adj[u])  
    cout << "-> " << v;
```

กราฟมีน้ำหนัก

```
for (auto it = adj[u].begin(); it!=adj[u].end(); it++)  
{  
    v = it -> first;  
    w = it -> second;  
    cout << " node " << v << " " << w;  
}
```

ปัญหาการหาเส้นทางในกราฟ

นิยาม กำหนดให้เมตริกประชิด (adjacency matrix) A แทนกราฟทิศทาง (Directed graph) หรือ digraph



(a) digraph

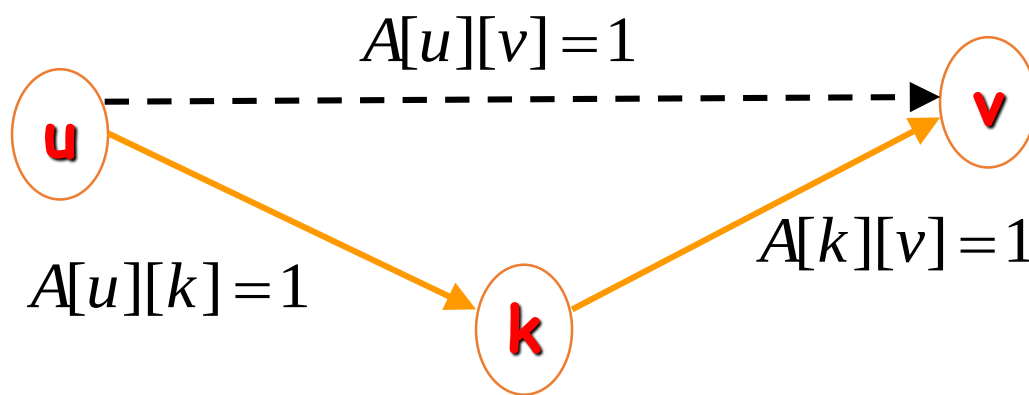
$$\begin{array}{c}
 \begin{array}{c} a \\ b \\ c \\ d \end{array}
 \begin{array}{cccc}
 a & b & c & d \\
 \left[\begin{array}{cccc}
 0 & 1 & 0 & 0 \\
 0 & 0 & 0 & 1 \\
 0 & 0 & 0 & 0 \\
 1 & 0 & 1 & 0
 \end{array} \right]
 \end{array}
 \end{array}$$

(b) adjacency matrix

ต้องการหาทุกเส้นทาง (path) ที่เป็นไปได้ในกราฟนี้ จะมีวิธีการอย่างไร ?? DFS/BFS ??

อัลกอริทึม Warshall

- เป็นอัลกอริทึมค้นหาเส้นทางที่เป็นไปทั้งหมดในกราฟไม่มีน้ำหนักและสามารถพัฒนาง่าย
- เริ่มต้นจะหาเส้นเชื่อมโดยตรง (directed edge) ระหว่างเวอร์เท็กซ์ (vertex) ในกราฟ (path = 1)
จากนั้นจะเริ่มหาเส้นทางที่ยาวขึ้นโดยใช้เวอร์เท็กซ์ intermediate
- ผลลัพธ์ของอัลกอริทึมถูกเก็บในรูปอาร์เรย์สองมิติ เรียกว่าเมตริกซ์ transitive relation

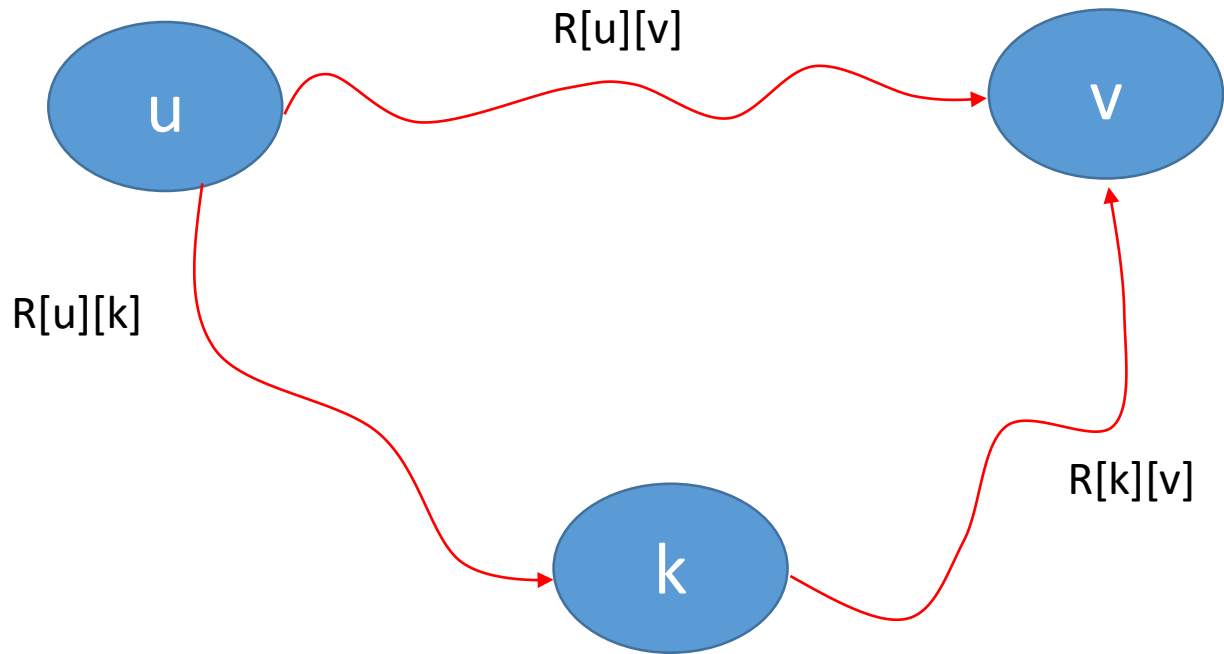


ตรวจสอบว่ามีเส้นทางจาก u ไป v หรือไม่

1. มีเอจด์จาก u ไป v หรือ
2. มีเวอร์เท็กซ์ k ที่มีเส้นทางจาก u ไป k และ จาก k ไป v

$$R[u][v] = A[u][v] \vee (A[u][k] \wedge A[k][v])$$

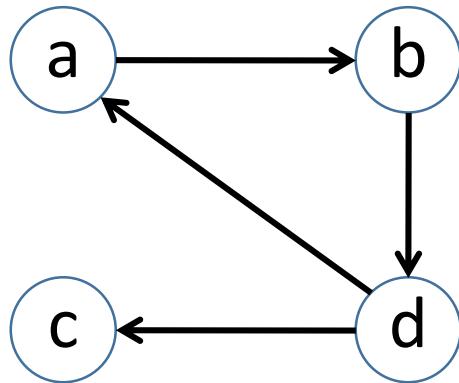
อัลกอริทึม Warshall



ปรับปรุงค่าในเมตริกซ์ R ด้วยกฎการปรับปรุงด้านล่าง โดยที่ k เป็นเวอร์เท็กซ์ intermediate ซึ่งรันจาก 1 ไปถึง n

$$R^{(t+1)}[u][v] = \begin{cases} 1 & R^t[u][v] = 1 \text{ OR } (R^t[u][k] = 1 \text{ and } R^t[k][v] = 1) \\ 0 & \text{Otherwise} \end{cases}$$

อัลกอริทึม Warshall



Digraph

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	0	1	0	0
<i>b</i>	0	0	0	1
<i>c</i>	0	0	0	0
<i>d</i>	1	0	1	0

Adjacency matrix

(A)

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	1	1	1	1
<i>b</i>	1	1	1	1
<i>c</i>	0	0	0	0
<i>d</i>	1	1	1	1

Transitive relation matrix

(R)

เมตริกซ์ Transitive relation R จะแสดงเส้นทางที่เกิดขึ้นทั้งหมดใน digraph

อัลกอริทึม Warshall

- Warshall's algorithm เป็น iterative search ซึ่งจะสร้าง transitive relation matrix ขนาด $n \times n$ จาก adjacency matrix ของ digraph

$$R^{(0)} \rightarrow R^{(1)} \rightarrow \dots \rightarrow R^{(k-1)} \rightarrow R^{(k)} \rightarrow \dots \rightarrow R^{(n)}$$

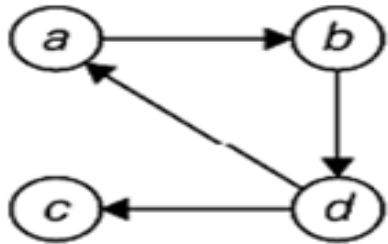
- $R^{(0)}$ คือ adjacency matrix ของกราฟ
- $R^{(1)}$ คือ เมตริกซ์ path ที่มี 1^{st} vertex เป็น intermediate vertex
- $R^{(2)}$ คือ เมตริกซ์ path ที่มี 1^{st} และ 2^{rd} vertices เป็น intermediate vertices
- เมตริกซ์สุดท้าย $R^{(n)}$ คือ เมตริกซ์ path ที่มี $1, 2, \dots, n$ vertices เป็น intermediate vertices ซึ่งก็คือผลลัพธ์ของอัลกอริทึม

การปรับปรุงค่าในเมตริกซ์ R

$$R^{(k-1)} = \begin{matrix} & j & k \\ \begin{matrix} k \\ i \end{matrix} & \begin{bmatrix} & & \\ & 1 & \\ 0 \rightarrow & & 1 \end{bmatrix} \end{matrix} \Rightarrow R^{(k)} = \begin{matrix} & j & k \\ \begin{matrix} k \\ i \end{matrix} & \begin{bmatrix} & & \\ & 1 & \\ & 1 & 1 \end{bmatrix} \end{matrix}$$

$$R^{(k)}[i,j] = R^{(k-1)}[i,j] \text{ or } (R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j])$$

ตัวอย่าง



$$R^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$R^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$R^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

$$R^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

$$R^{(4)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$



```
#include<stdio.h>
```

```
#define V 4
```

```
void Warshall(int g[][V])
```

```
{   for(int k=0; k < V; k++)           // intermediate vertex
```

```
    for(int u=0; u < V; u++)           // pair of u and v
```

```
        for(int v=0; v < V; v++)
```

```
            g[u][v] = g[u][v] | (g[u][k] & g[k][v]);
```

```
}
```

```
void main()
```

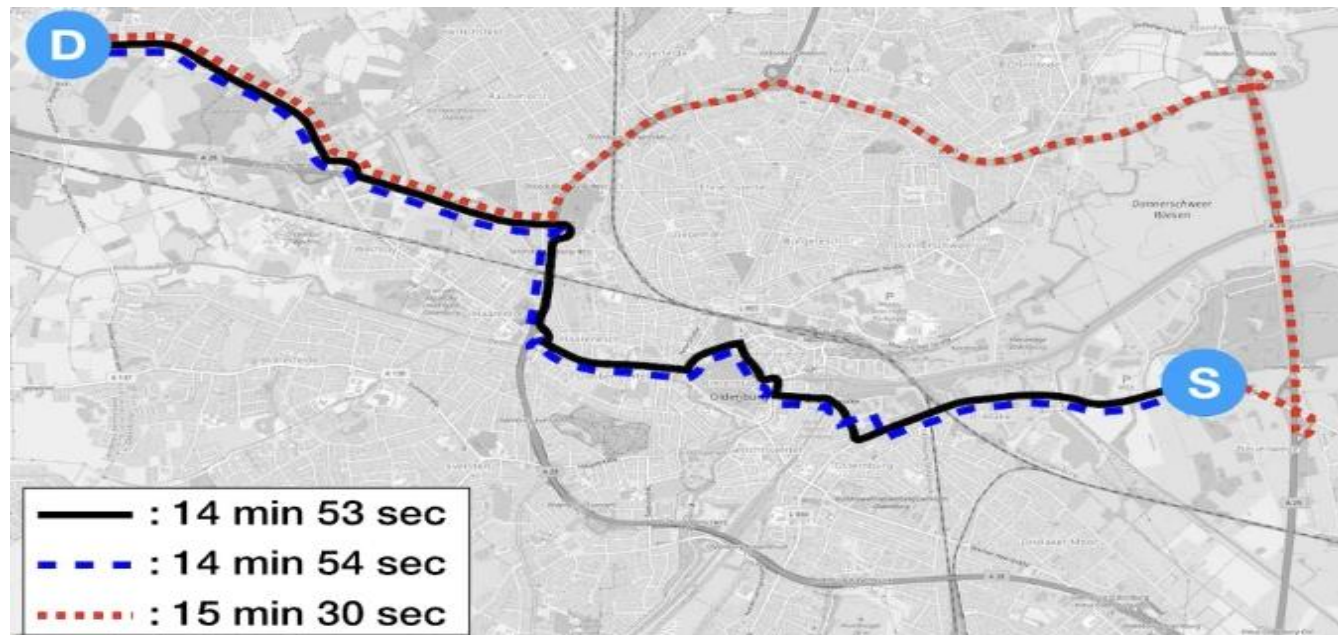
```
{   int graph[V][V] = { { 0, 1, 0, 0 },
                        { 0, 0, 0, 1 },
                        { 0, 0, 0, 0 },
                        { 1, 0, 1, 0 } };
```

```
    Warshall(graph);
```

```
}
```

ปัญหาเส้นทางที่สั้นที่สุด (Shortest path)

- ปัญหาเส้นทางสั้นที่สุด (shortest path) หมายถึงเส้นทางระหว่างเวอร์เท็กซ์ในกราฟที่ให้ความยาวรวมของเอจด์ (edge) น้อยที่สุด
- เป็นปัญหาของกราฟมีน้ำหนัก (weighted graph) เนื่องจากค่าน้ำหนักของเอจด์อาจแทนระยะทางระหว่างเวอร์เท็กซ์ได้ ดังนั้นเป็นการหาผลรวมที่น้อยที่สุดของเอจด์จาก u ไป v



Shortest path problem



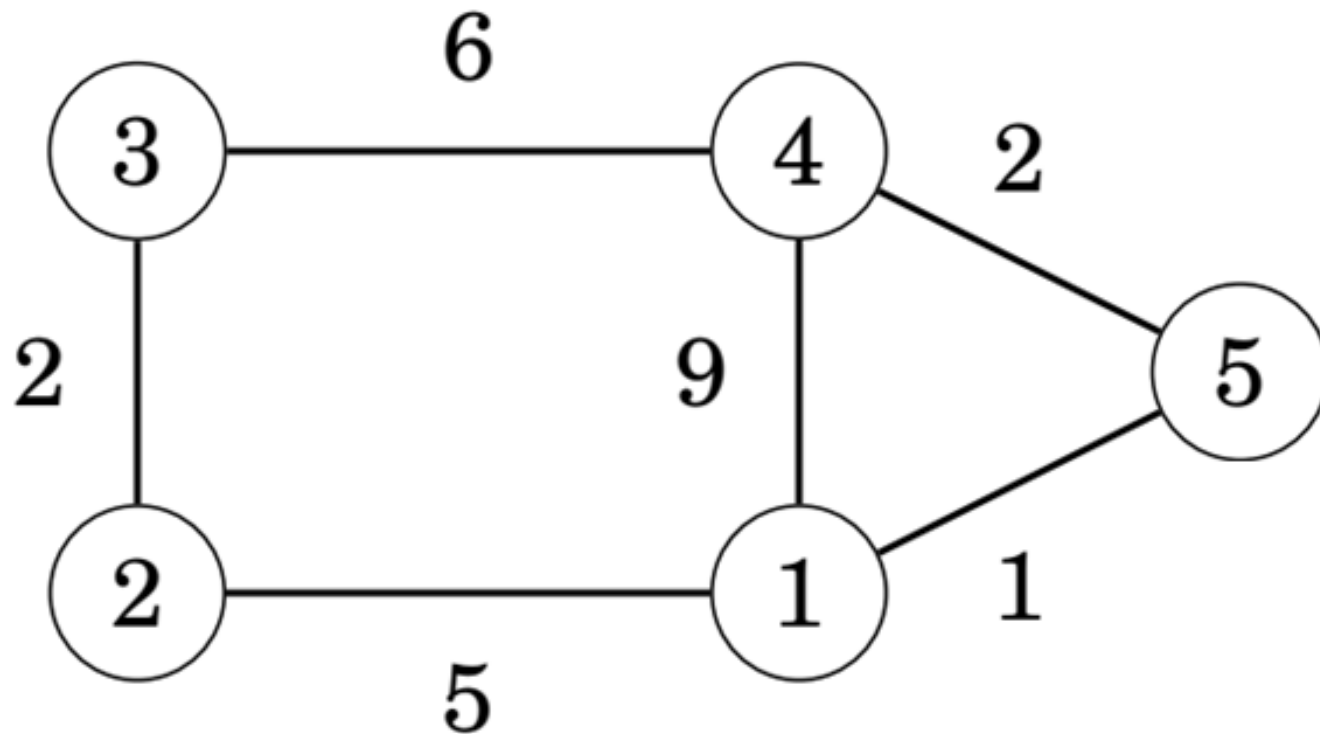
ปัญหาเส้นทางที่สั้นที่สุด (Shortest path)

ปัญหาการหาเส้นทางที่สั้นที่สุด มักสามารถแบ่งออกเป็นสองประเภท ได้แก่

1. การหาเส้นทางสั้นที่สุดโดยกำหนดจุดเริ่มต้น (single source) เรียกปัญหานี้ว่า
Single Source Shortest Path (SSSP)
2. การหาเส้นทางที่สั้นที่สุดที่จุดใดก็ได้ เรียกปัญหานี้ว่า
All-Pair Shortest Path (APSP)

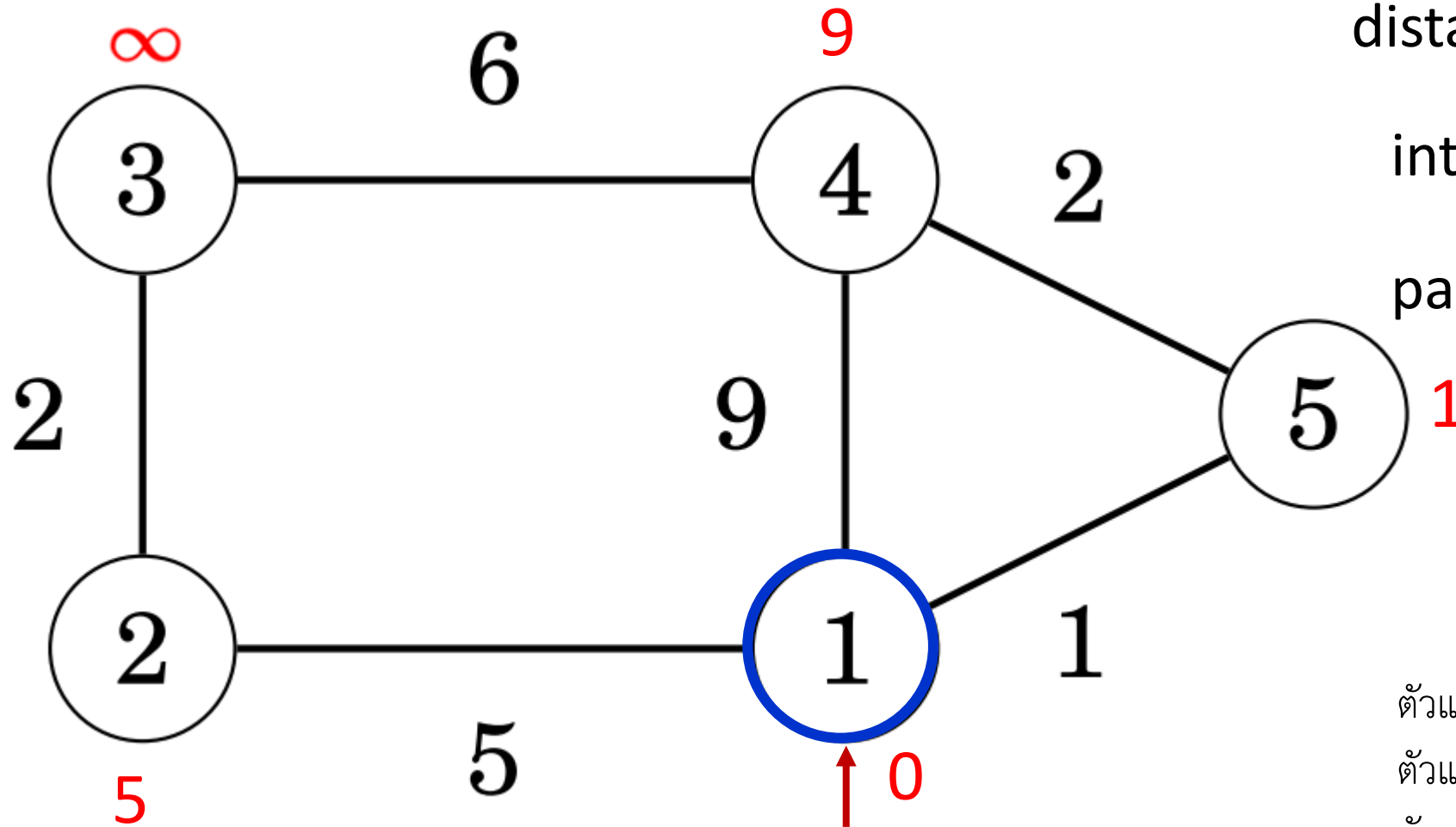
Dijkstra algorithm

กำหนดจุดเริ่มต้น (single source) ในกราฟ จากนั้นจะเริ่มแผ่ไปยังเอดจ์ (edge) รอบข้างที่มีความยาวสั้นที่สุด และแผ่ต่อไปโดยมีเงื่อนไขว่าเอดจ์จะถูกเลือกก็ต่อเมื่อระยะทางจาก source รวมกับน้ำหนักของเอดจ์น้อยที่สุด



ต้องการหาเส้นทางสั้นสุดจากเวอร์เท็กซ์
หมายเลข **1** ไปยังทุกเวอร์เท็กซ์ในกราฟ

Dijkstra algorithm



distance

	1	2	3	4	5
distance	0	5	∞	9	1

intree

	1	2	3	4	5
intree	1	0	0	0	0

parent

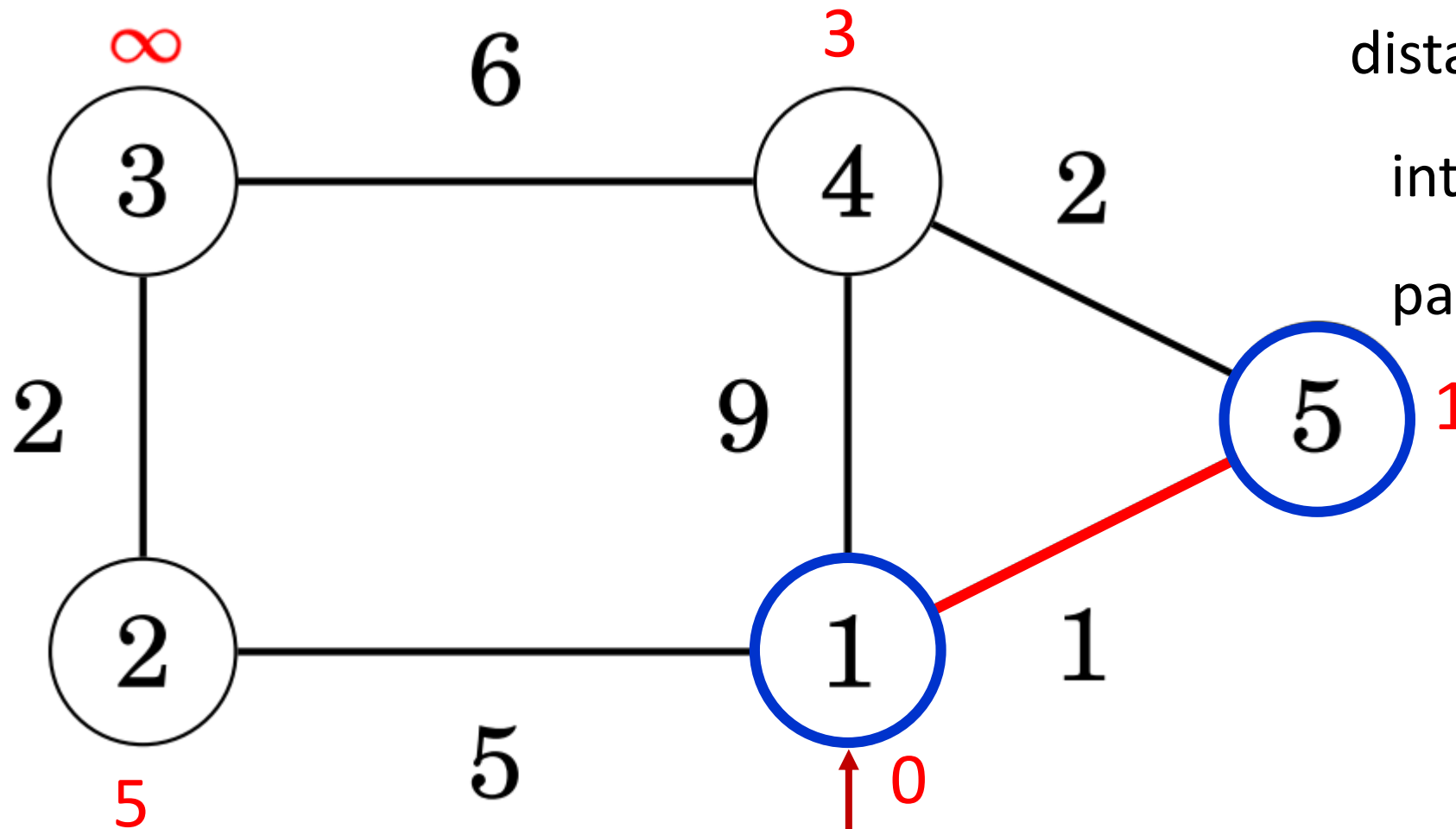
	1	2	3	4	5
parent	-1	-1	-1	-1	-1

ตัวแปร distance เก็บระยะทางจาก source

ตัวแปร intree เก็บสถานะการเลือก vertex

ตัวแปร parent เก็บ path ของต้นไม้

Dijkstra algorithm



distance

1	2	3	4	5
0	5	∞	3	1

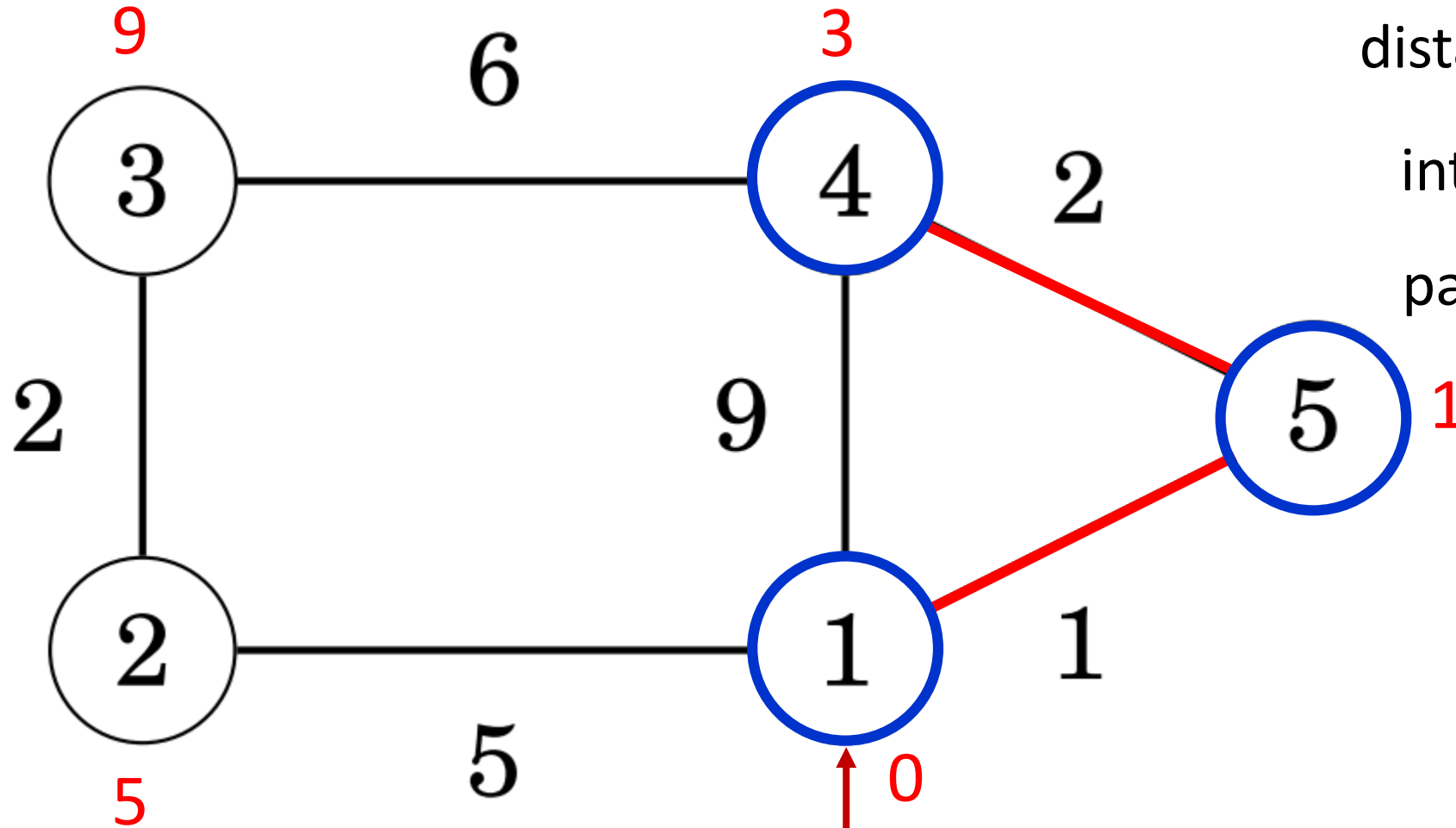
intree

1	2	3	4	5
1	0	0	0	1

parent

1	2	3	4	5
-1	-1	-1	-1	1

Dijkstra algorithm



distance

1	2	3	4	5
0	5	9	3	1

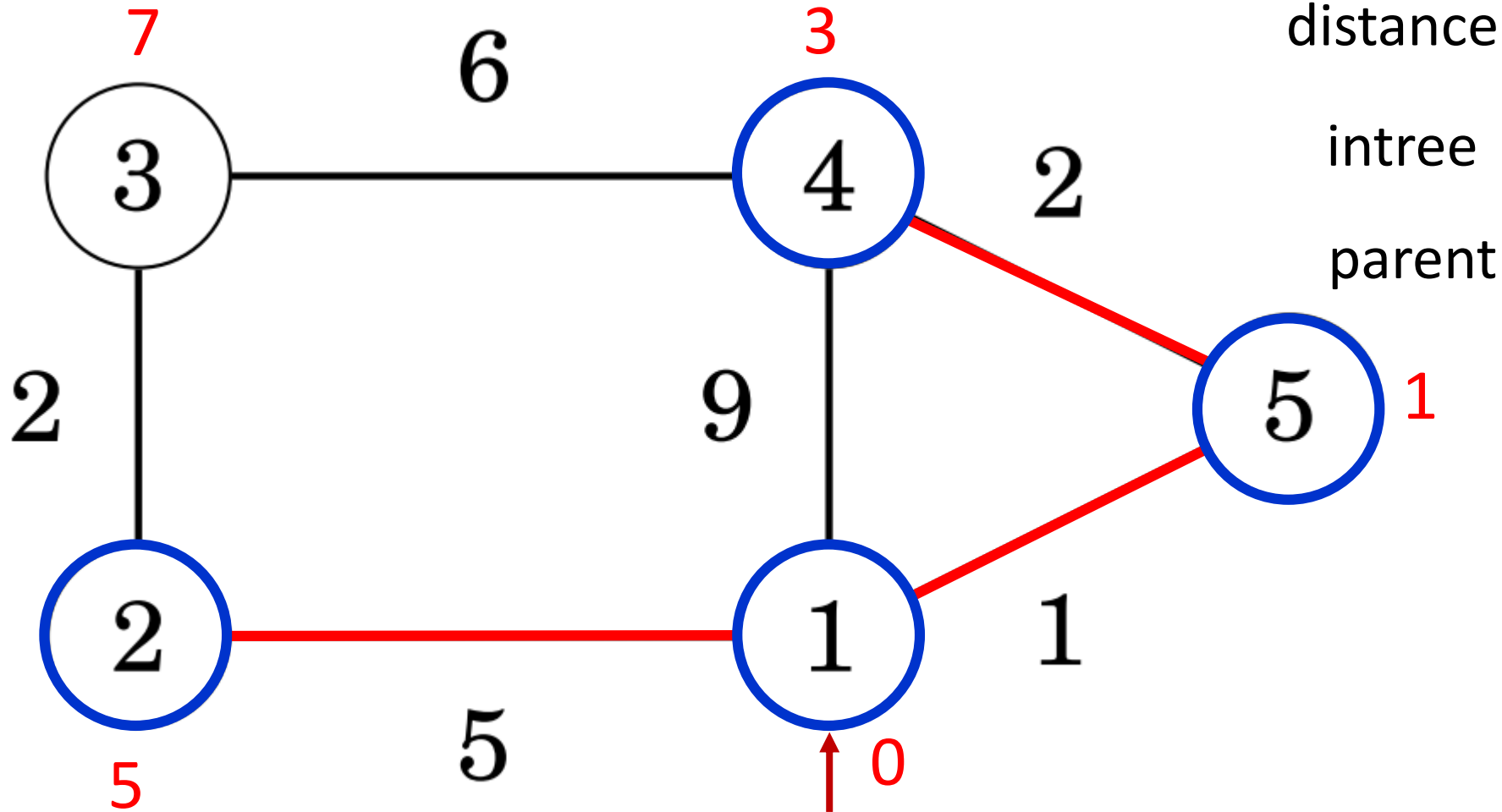
intree

1	2	3	4	5
1	0	0	1	1

parent

1	2	3	4	5
-1	-1	-1	5	1

Dijkstra algorithm



distance

1	2	3	4	5
0	5	7	3	1

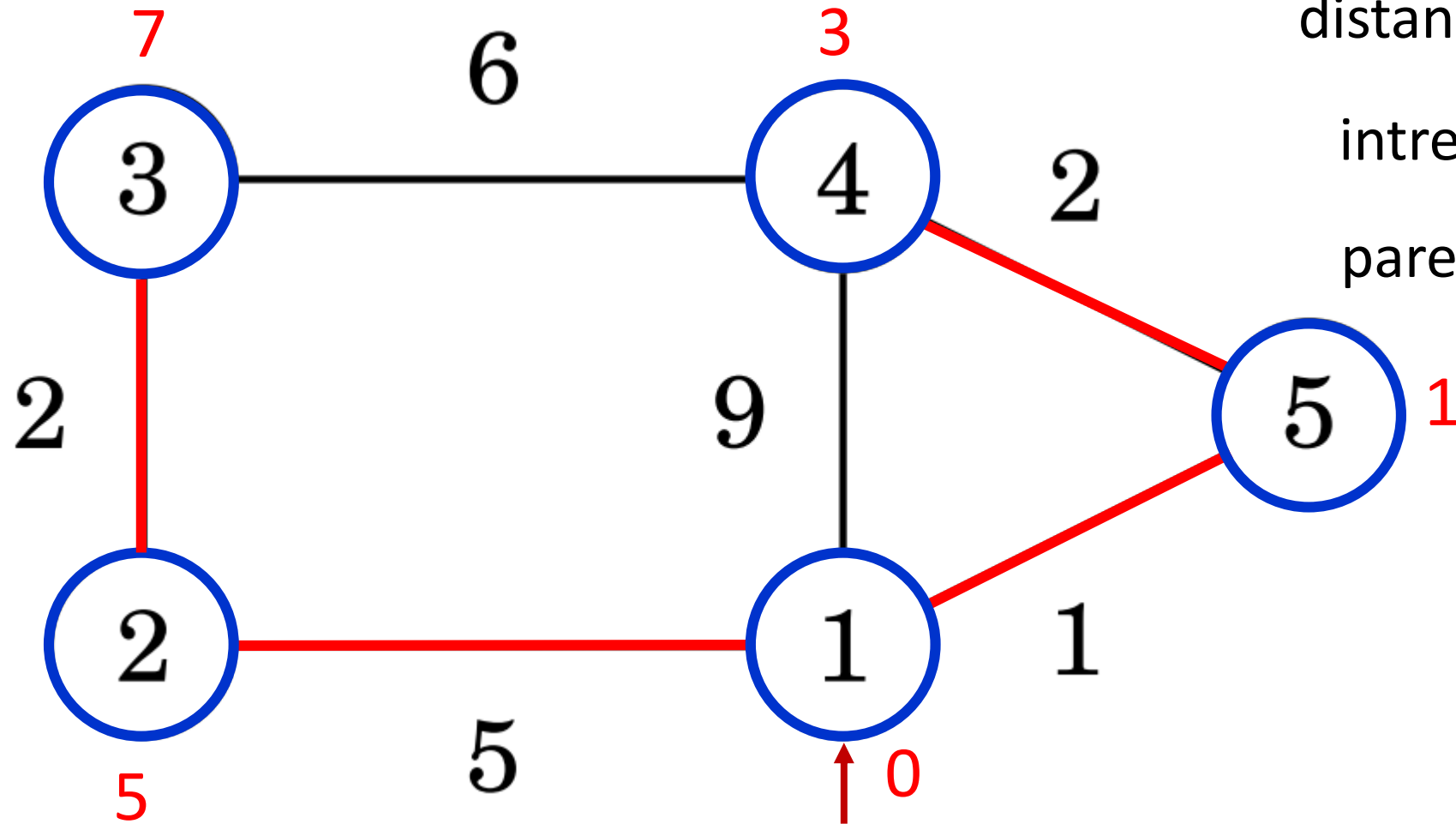
intree

1	2	3	4	5
1	1	0	1	1

parent

1	2	3	4	5
-1	1	-1	5	1

Dijkstra algorithm



distance

1	2	3	4	5
0	5	7	3	1

intree

1	2	3	4	5
1	1	1	1	1

parent

1	2	3	4	5
-1	1	2	5	1

Dijkstra algorithm

```
#define inf 50000
```

```
void dijkstra(int graph[][V], int src)
```

```
{ int dist[V], inTree[V], parent[V];
```

```
    for (int i = 0; i < V; i++)    // initial values
        dist[i] = inf, parent[i] = -1, inTree[i] = -1;
```

```
    dist[src] = 0;
```

```
    for(int i = 0; i < V-1; i++)
```

```
        {int u = minDistance(dist, inTree);
```

```
        inTree[u] = 1;
```

```
        for (int v = 0; v < V; v++)
```

```
            if (inTree[v] == false && graph[u][v] > 0 && dist[u] + graph[u][v] < dist[v])
```

```
                { dist[v] = dist[u] + graph[u][v];
```

```
                parent[v] = u;
```

```
            }
```

```
        }
```

```
}
```

```
int minDistance(int dist[], int inTree[])
{ int min = inf, min_index;
```

```
    for(int v = 0; v < V; v++)
```

```
        if (inTree[v] == false && dist[v] <= min)
```

```
            { min = dist[v];
```

```
            min_index = v;
```

```
            }
```

```
    return min_index;
```

```
}
```

```
// find node u with minimum distance
```

```
// add u into the tree
```

```
// update all minimum weights
```

```
// update parent
```



```
#include <iostream>
#include <queue>
```

```
using namespace std;
```

```
void main()
{
    priority_queue<int, vector<int>, greater<int> > pq;

    pq.push(10);
    pq.push(30);
    pq.push(20);
    pq.push(5);
    pq.push(1);

    while (!pq.empty()) {
        cout << '\t' << pq.top();
        pq.pop();
    }
}
```



```
#include<bits/stdc++.h>
using namespace std;
```

```
typedef pair<int, int> iPair;
```

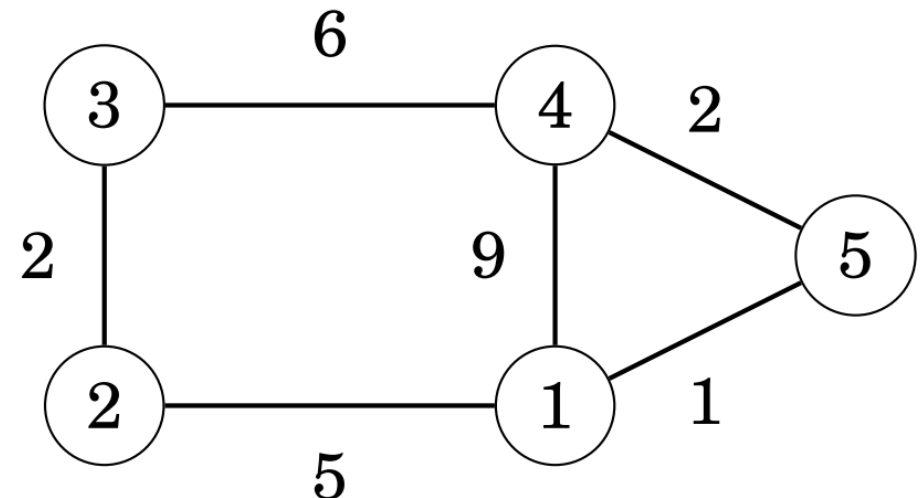
```
int main()
{   int V = 5;
    vector<iPair > adj[V];

    addEdge(adj, 1, 2, 5);
    addEdge(adj, 2, 1, 5);
    addEdge(adj, 1, 4, 9);
    addEdge(adj, 4, 1, 9);
    addEdge(adj, 1, 5, 1);
    addEdge(adj, 5, 1, 1);
    addEdge(adj, 4, 5, 2);
    addEdge(adj, 5, 4, 2);
    addEdge(adj, 3, 4, 6);
    addEdge(adj, 4, 3, 6);
    addEdge(adj, 2, 3, 2);
    addEdge(adj, 3, 2, 3);

    Dijkstra(adj, V, 1);
}
```

Improve Dijkstra with adj. list and Priority Queue

```
void addEdge(vector <pair<int, int> > adj[],
int u, int v, int wt)
{
    adj[u].push_back(make_pair(v, wt));
    adj[v].push_back(make_pair(u, wt));
}
```





```
void Dijkstra(vector<pair<int,int> > adj[], int V, int src)
{
    priority_queue< iPair, vector <iPair> , greater<iPair> > pq;
    vector<int> dist(V, INF);
```

```
    pq.push({0, src});                // push start vertex src
    dist[src] = 0;                     // initial distance of src

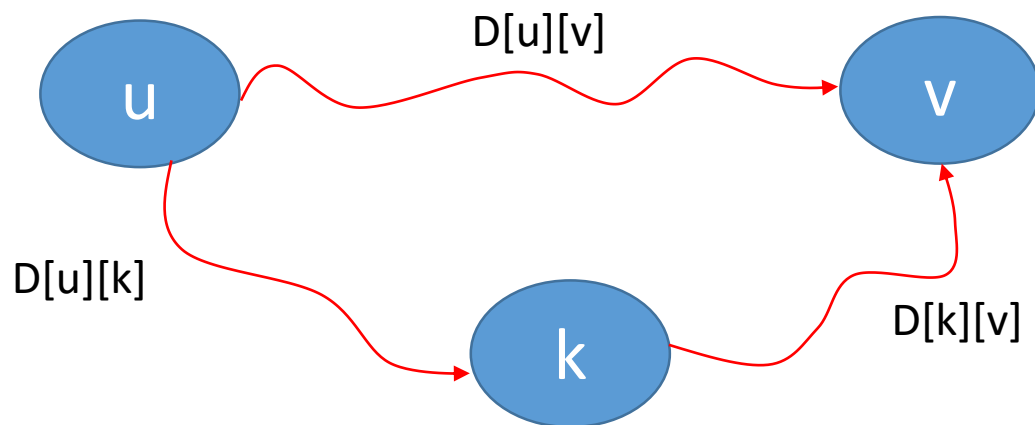
    while (!pq.empty())
    { int u = pq.top().second;         // get u from pq.
      pq.pop();

      for (auto x : adj[u])
      { int v = x.first;               // get v and weight from adj list.
        int weight = x.second;

        if (dist[v] > dist[u] + weight)
        { dist[v] = dist[u] + weight;  // Update weight of v
          pq.push( {dist[v], v} );    // push v into pq.
        }
      }
    }
}
```

Floyd-Warshall algorithm

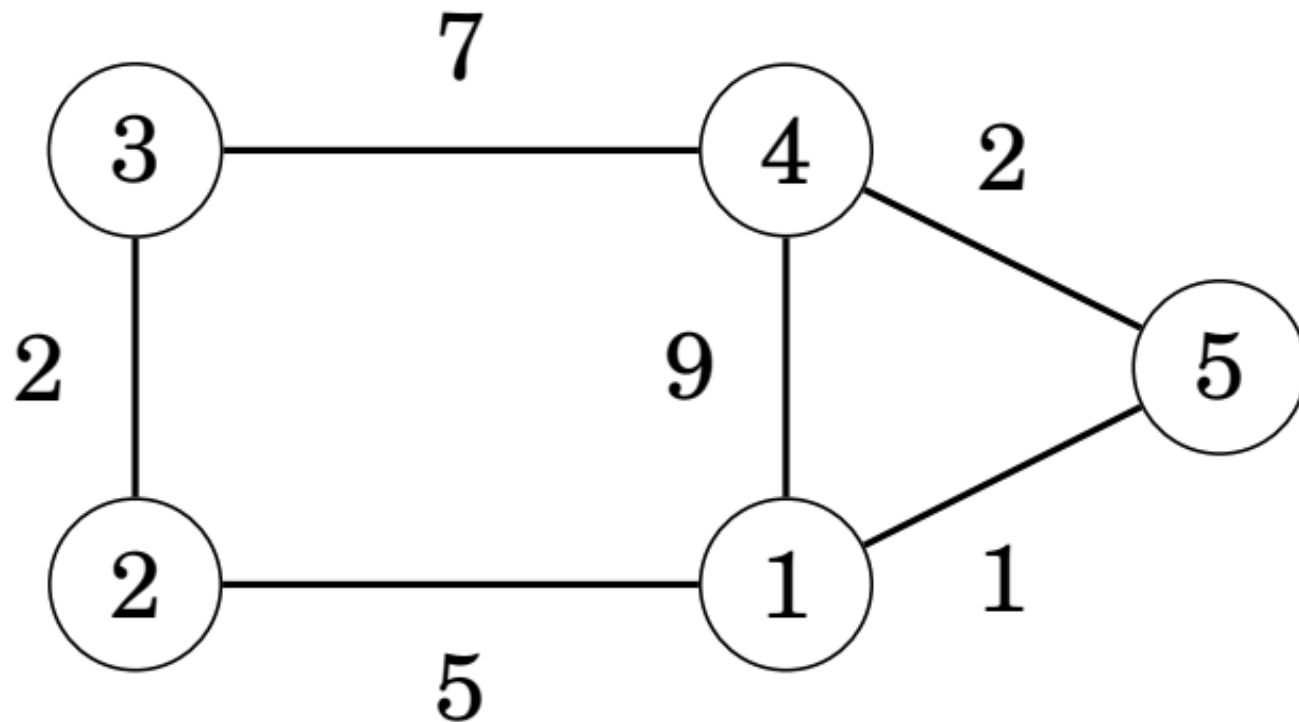
- อัลกอริทึมทางเลือกสำหรับการหาเส้นทางสั้นที่สุดในกราฟมีน้ำหนัก
- แตกต่างจากอัลกอริทึม Dijkstra อัลกอริทึมนี้จะค้นหาทุกเส้นทางสั้นที่สุดระหว่างทุกคู่เวอร์เท็กซ์ด้วยการรันอัลกอริทึมเพียงครั้งเดียว เรียกปัญหานี้ว่า All pair shortest path
- ปรับปรุงอัลกอริทึม Warshall เพื่อให้สามารถบันทึกระยะทางสั้นสุดระหว่างคู่เวอร์เท็กซ์ลงในอาร์เรย์ 2 มิติ เรียกเมตริกซ์ใหม่นี้ว่า distance matrix (D)



$$D[u][v] = \min \{ D[u][v], D[u][k] + D[k][v] \}$$

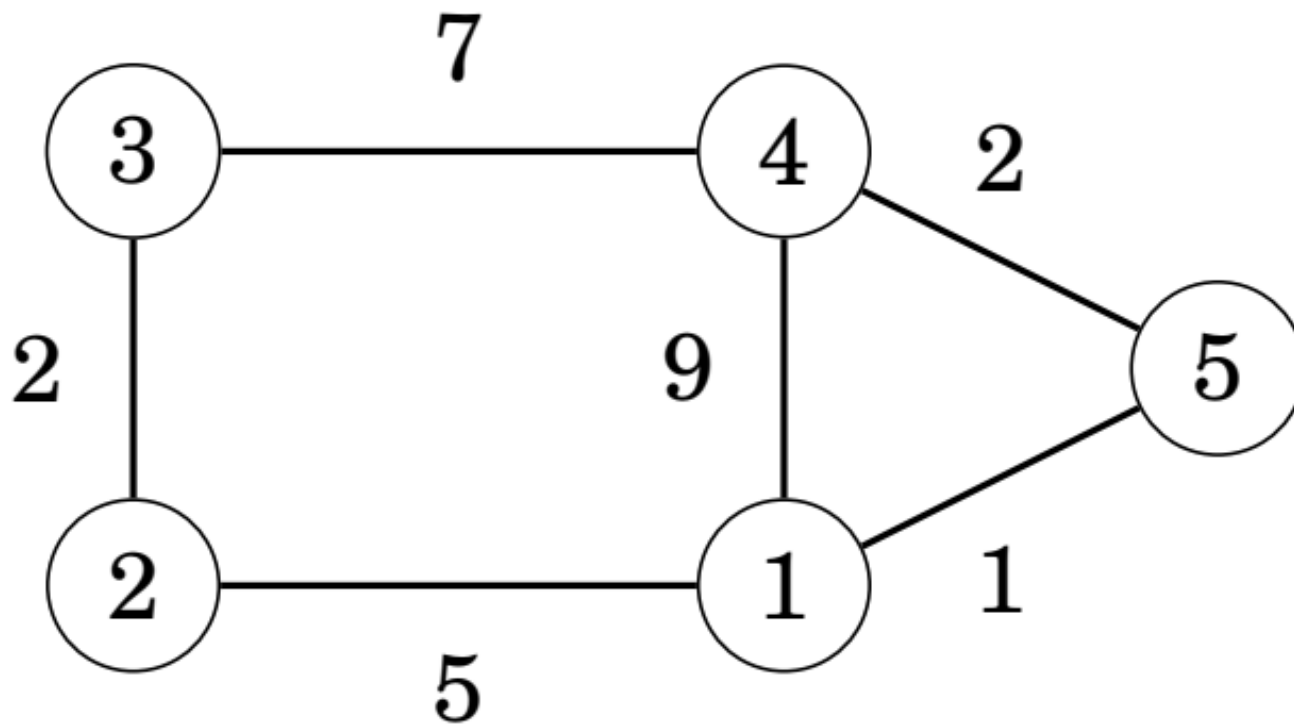
โดยที่ k เป็น intermediate vertex

Floyd-Warshall algorithm



	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

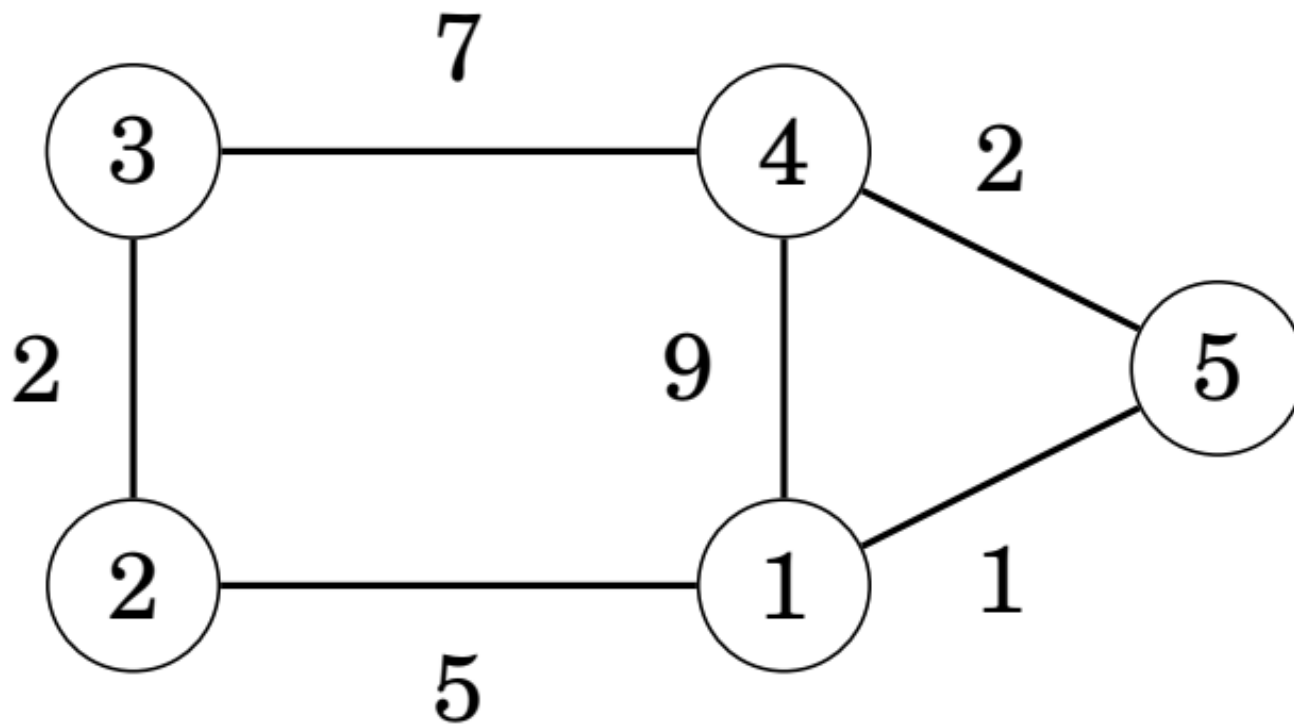
Floyd-Warshall algorithm



	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

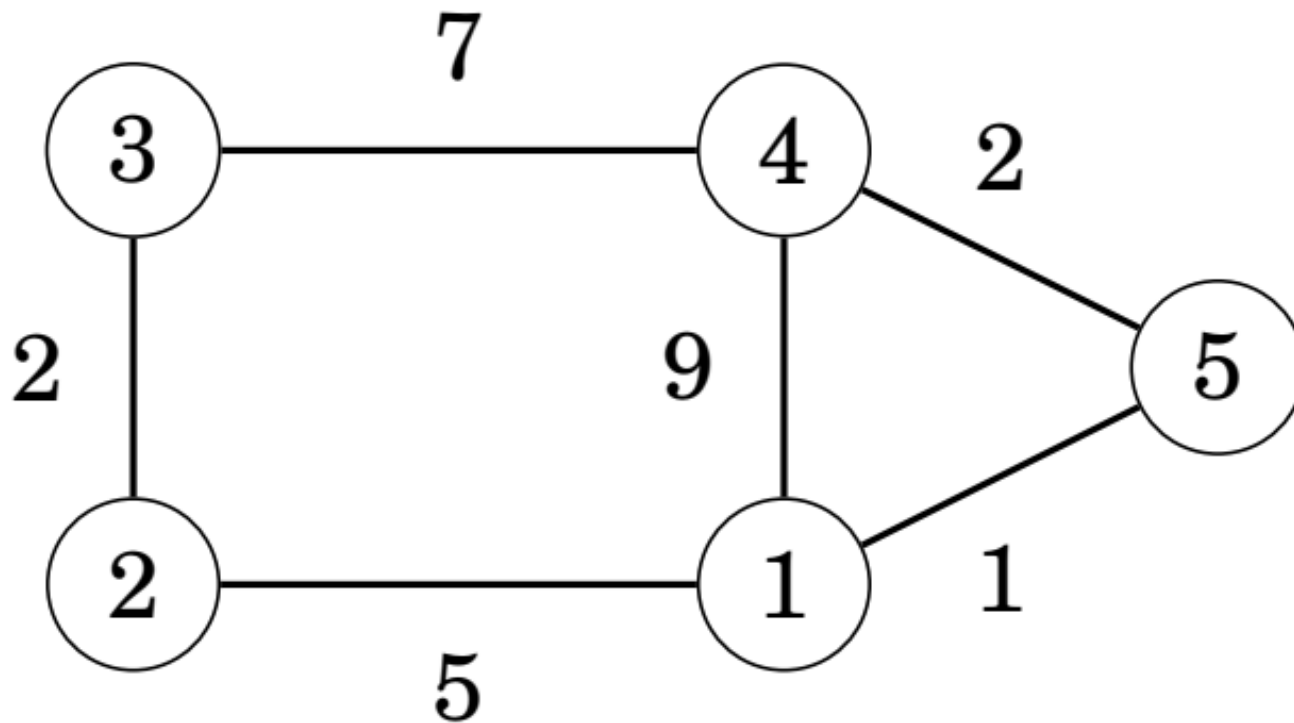
Floyd-Warshall algorithm



	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

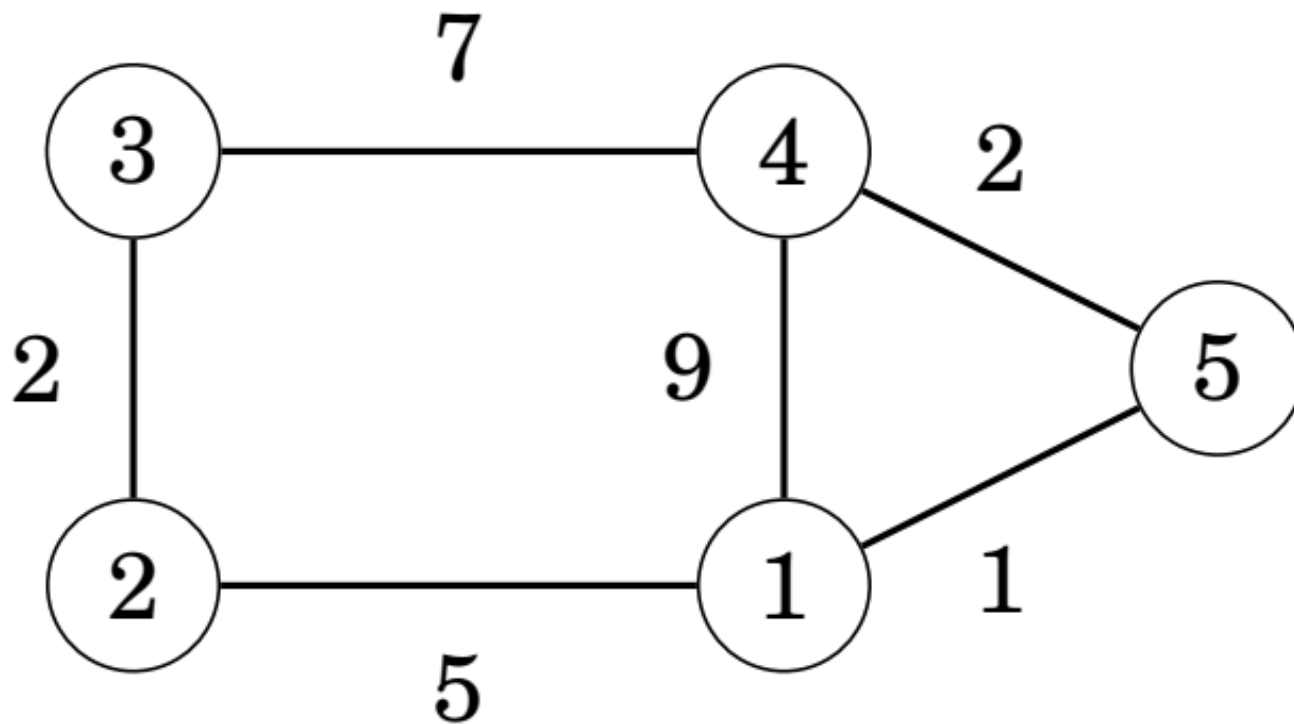
Floyd-Warshall algorithm



	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

Floyd-Warshall algorithm



	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

Floyd-Warshall algorithm

```
void Floyd(int dis[][V])
{
    for(int k=0; k < V; k++)
    {
        for(int u=0; u < V; u++)
        {
            for(int v=0; v < V; v++)
            {
                if (dis[u][v] > dis[u][k] + dis[k][v]) {
                    dis[u][v] = dis[u][k] + dis[k][v];
                }
            }
        }
    }
}
```

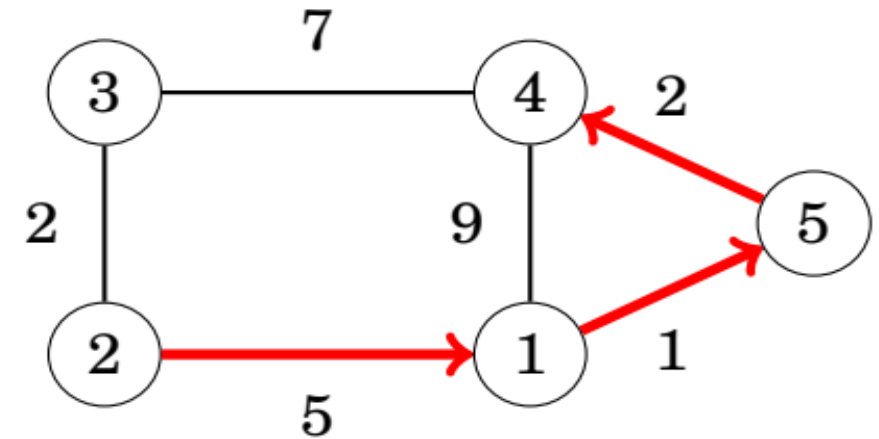
```
void main()
{
    int graph[][V] = { {0, 3, inf, 7},
                        {8, 0, 2, inf},
                        {5, inf, 0, 1},
                        {2, inf, inf, 0}};

    Floyd(graph);
}
```

เส้นทางที่สั้นสุด

- ผลลัพธ์ของอัลกอริทึม Floyd-Warshall คือระยะทางสั้นที่สุดในทุกคู่เวอร์เท็กซ์ เช่น เส้นทางสั้นที่สุดจาก 2 ไป 4 เท่ากับ 8
- บันทึก shortest path อย่างไร

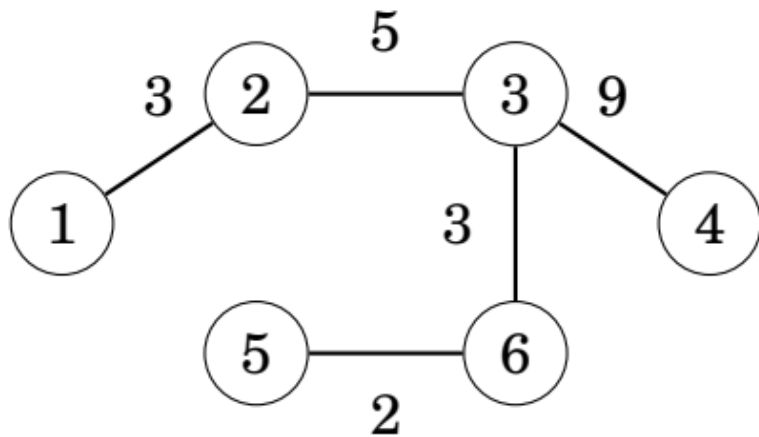
2 -> 1 -> 5 -> 4



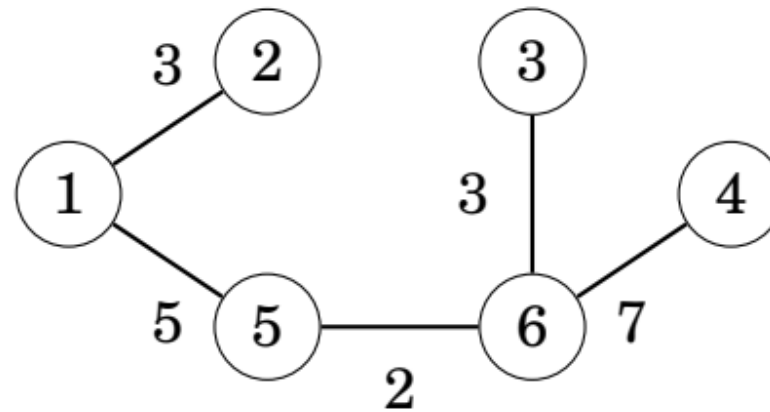
	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

การหาต้นไม้แผ่ทั่วที่เล็กสุด (Minimum Spanning Tree)

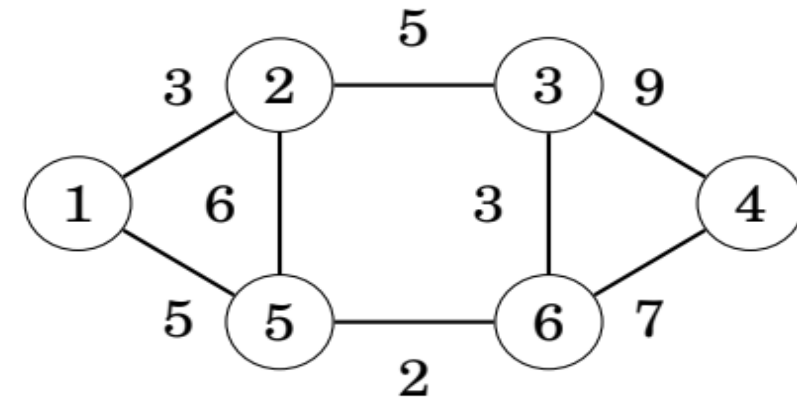
- ต้นไม้แผ่ทั่ว (Spanning tree) ของกราฟ หมายถึง acyclic graph ซึ่งเชื่อมโยงทุกเวอร์เท็กซ์ด้วยเอจด์บางเส้น ($|V| - 1$)
- ต้นไม้แผ่ทั่วที่เล็กสุด (Minimum spanning tree) คือต้นไม้แผ่ทั่วที่ผลรวมของเอจด์มีค่าน้อยที่สุด



$$3+5+9+3+2 = 22$$



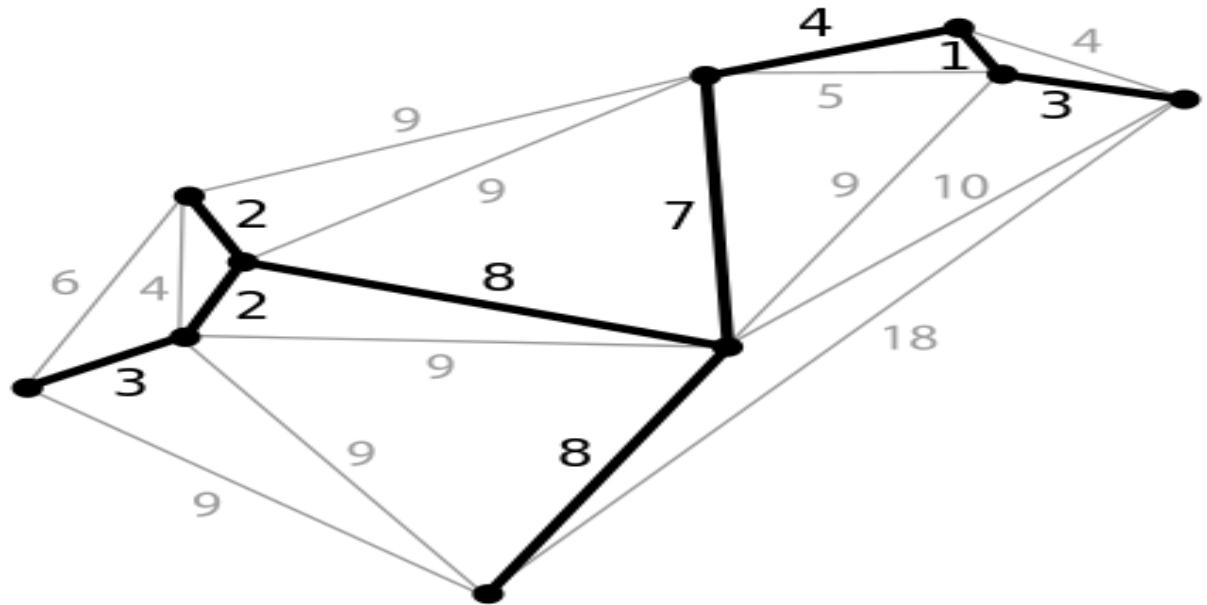
$$3+5+2+3+7 = 20$$





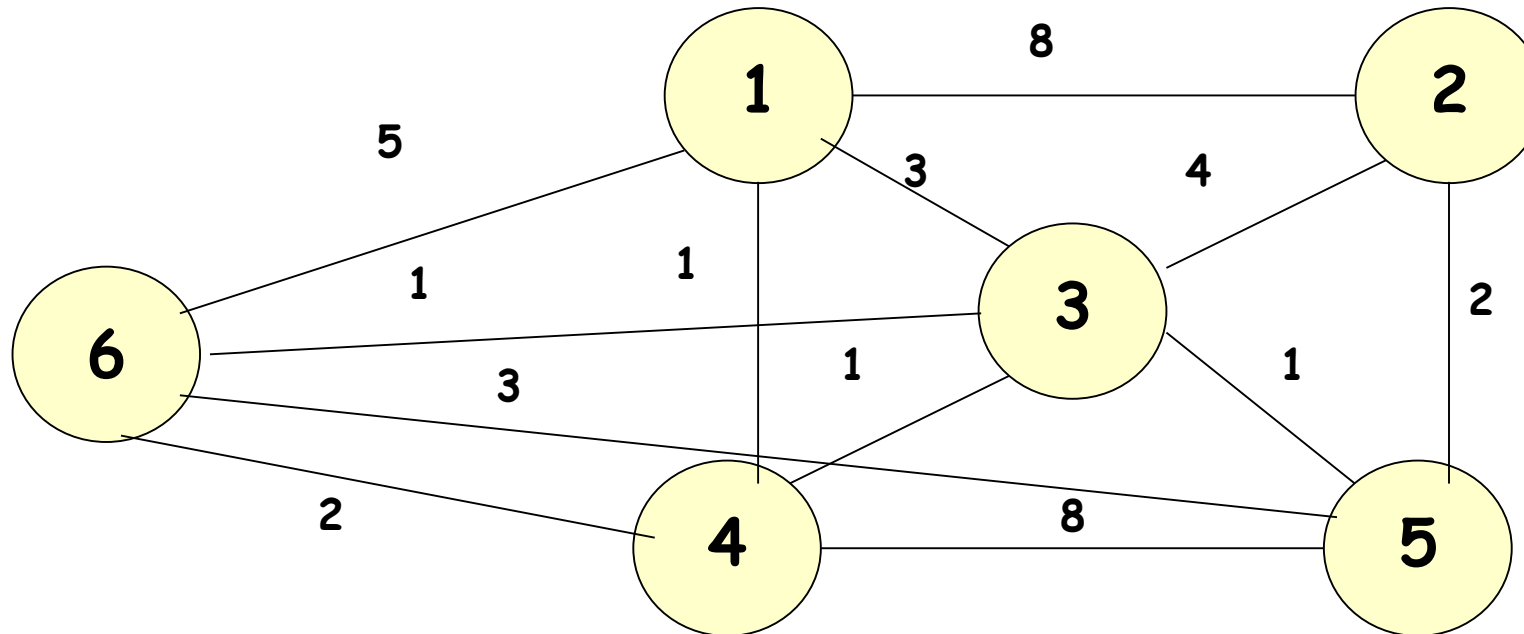
การหาต้นไม้แผ่ทั่วที่เล็กสุด (Minimum Spanning Tree)

- MST ในกราฟสามารถมีได้มากกว่า 1 คำตอบ
- อัลกอริทึม Brute force เพื่อค้นหา minimum spanning tree ในกราฟ อาจไม่เหมาะสม เนื่องจาก ต้นไม้แผ่ทั่วอาจมีจำนวนมากเกินกว่าที่จะสร้างออกมาได้ทั้งหมด (exponential time)
- หลักการของอัลกอริทึม greedy จึงเหมาะสมกว่า (Polynomial time)
 - อัลกอริทึม Prim
 - อัลกอริทึม Kruskal



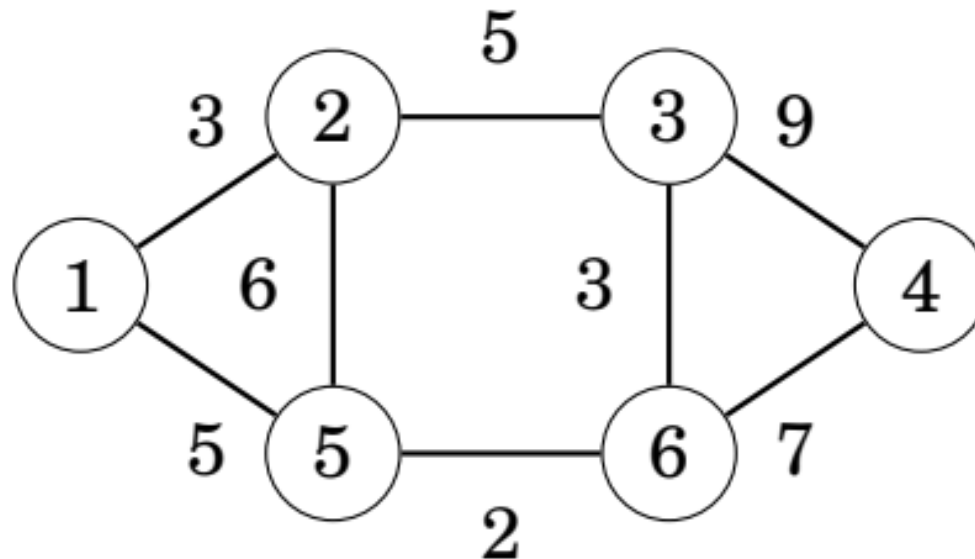
โจทย์

จงหา minimum spanning tree ของกราฟด้านล่าง

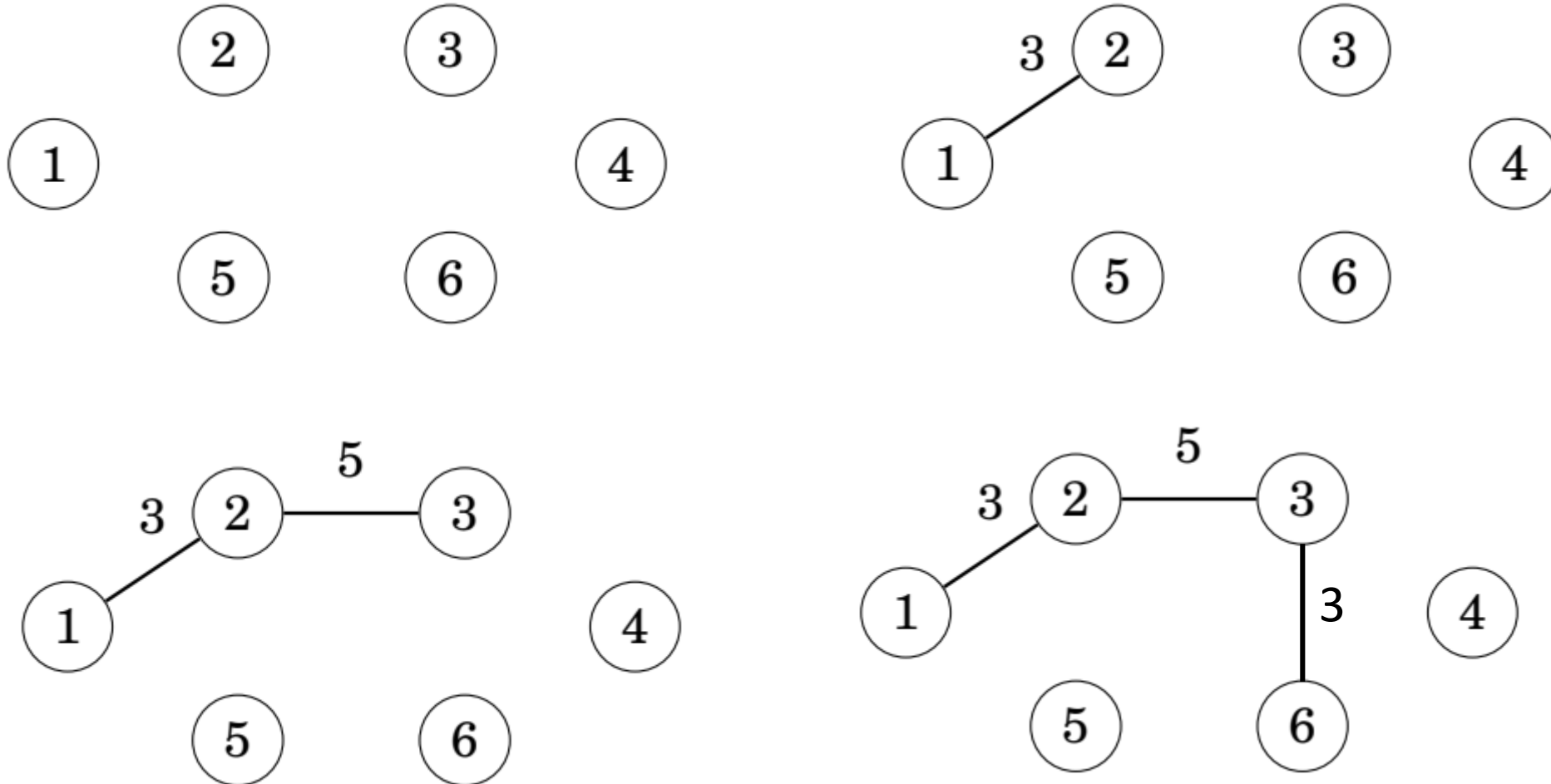
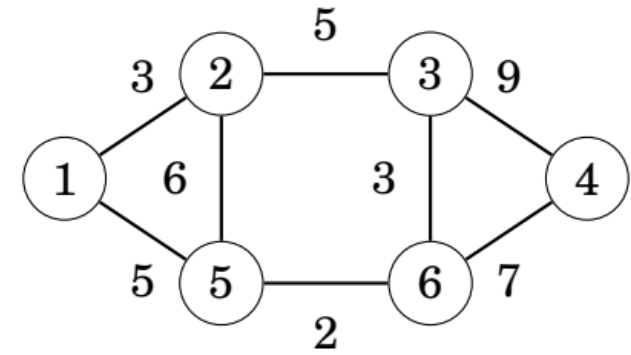


Prim's algorithm

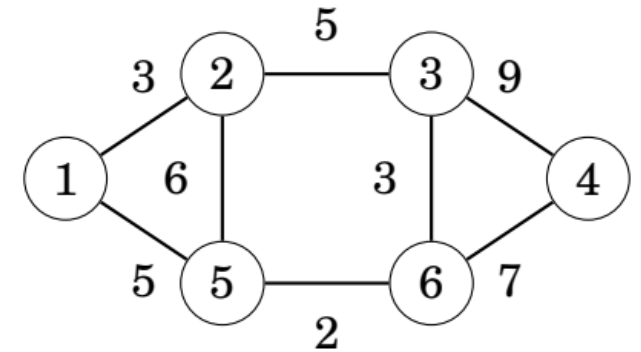
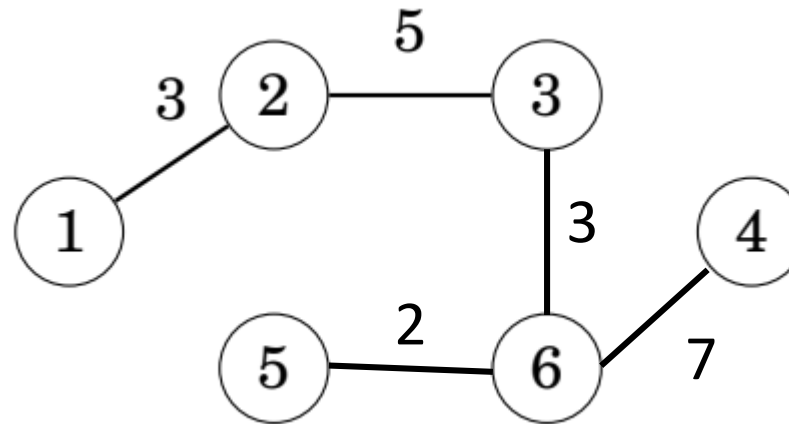
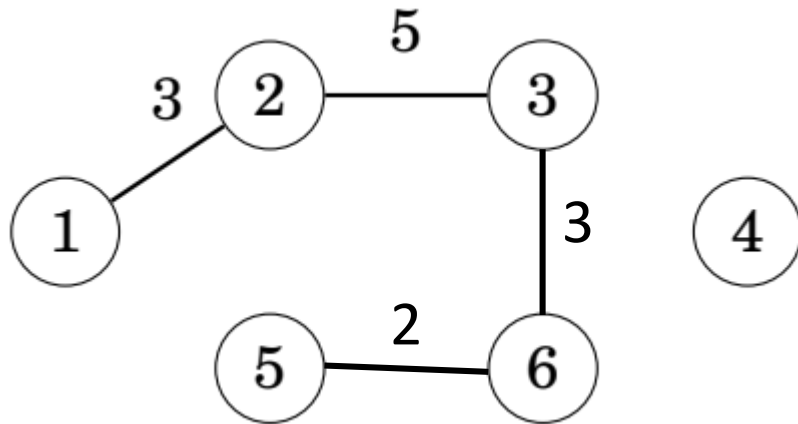
- เป็นอัลกอริทึมสำหรับค้นหา MST ที่คล้ายคลึงกับอัลกอริทึม Dijkstra
- เริ่มต้นจะต้องกำหนดเวอร์เท็กซ์ในกราฟเพื่อสร้าง MST จากนั้นอัลกอริทึมจะเริ่มเลือกเอจด์ที่มีค่าน้อยสุด (minimum-weight edge) มาเพิ่มให้กับ MST
- เมื่อเลือกครบทุกเวอร์เท็กซ์ ($|V| - 1$) อัลกอริทึมก็จะหยุดทำงาน



Prim's algorithm



Prim's algorithm



MST
 $3+5+2+3+7 = 20$



```
#include<bits/stdc++.h>
using namespace std;
```

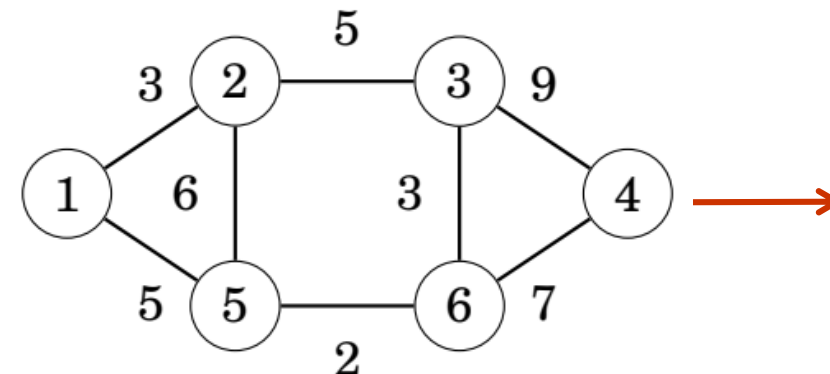
```
typedef pair<int, int> iPair;
```

```
int main()
{   int V = 6;
    vector<iPair > adj[V];
```

```
    addEdge(adj, 1, 2, 3);
    addEdge(adj, 1, 5, 5);
    addEdge(adj, 2, 5, 6);
    addEdge(adj, 2, 3, 5);
    addEdge(adj, 5, 6, 2);
    addEdge(adj, 3, 6, 3);
    addEdge(adj, 3, 4, 9);
    addEdge(adj, 4, 6, 7);
```

```
    PrimMST(adj, V, 1);
```

```
}
```



```
void addEdge(vector <pair<int, int> > adj[], int u,
int v, int wt)
{
    adj[u].push_back(make_pair(v, wt));
    adj[v].push_back(make_pair(u, wt));
}
```



```
primMST(vector<pair<int,int> > adj[], int V, int src)
{ priority_queue< iPair, vector <iPair> , greater<iPair> > pq;    // Priority Queue

  vector<int> key(V, INF);    // keep distance of vertex
  vector<int> parent(V, -1);  // keep path of MST
  vector<bool> inMST(V, false); // keep status of vertex

  pq.push(make_pair(0, src));
  key[src] = 0;
  while (!pq.empty())
  { int u = pq.top().second;    // get u from pq.
    pq.pop();

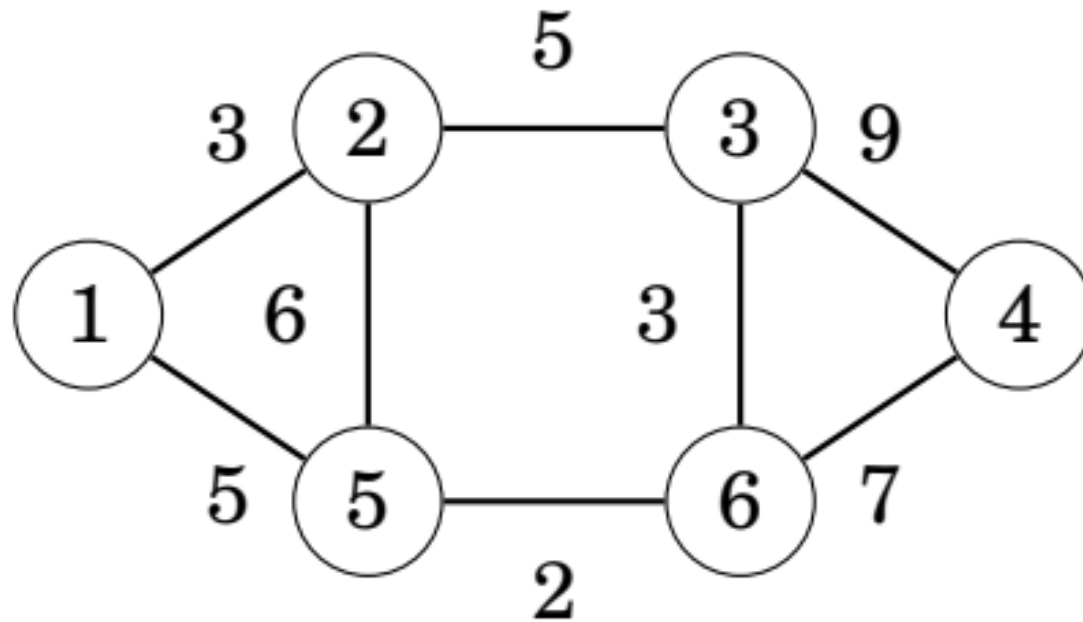
    if(inMST[u] == true) continue;
    inMST[u] = true;           // add u into MST

    list< pair<int, int> >::iterator i;
    for (i = adj[u].begin(); i != adj[u].end(); ++i)
    { int v = (*i).first;      //get v and weight from adj list of u
      int weight = (*i).second;

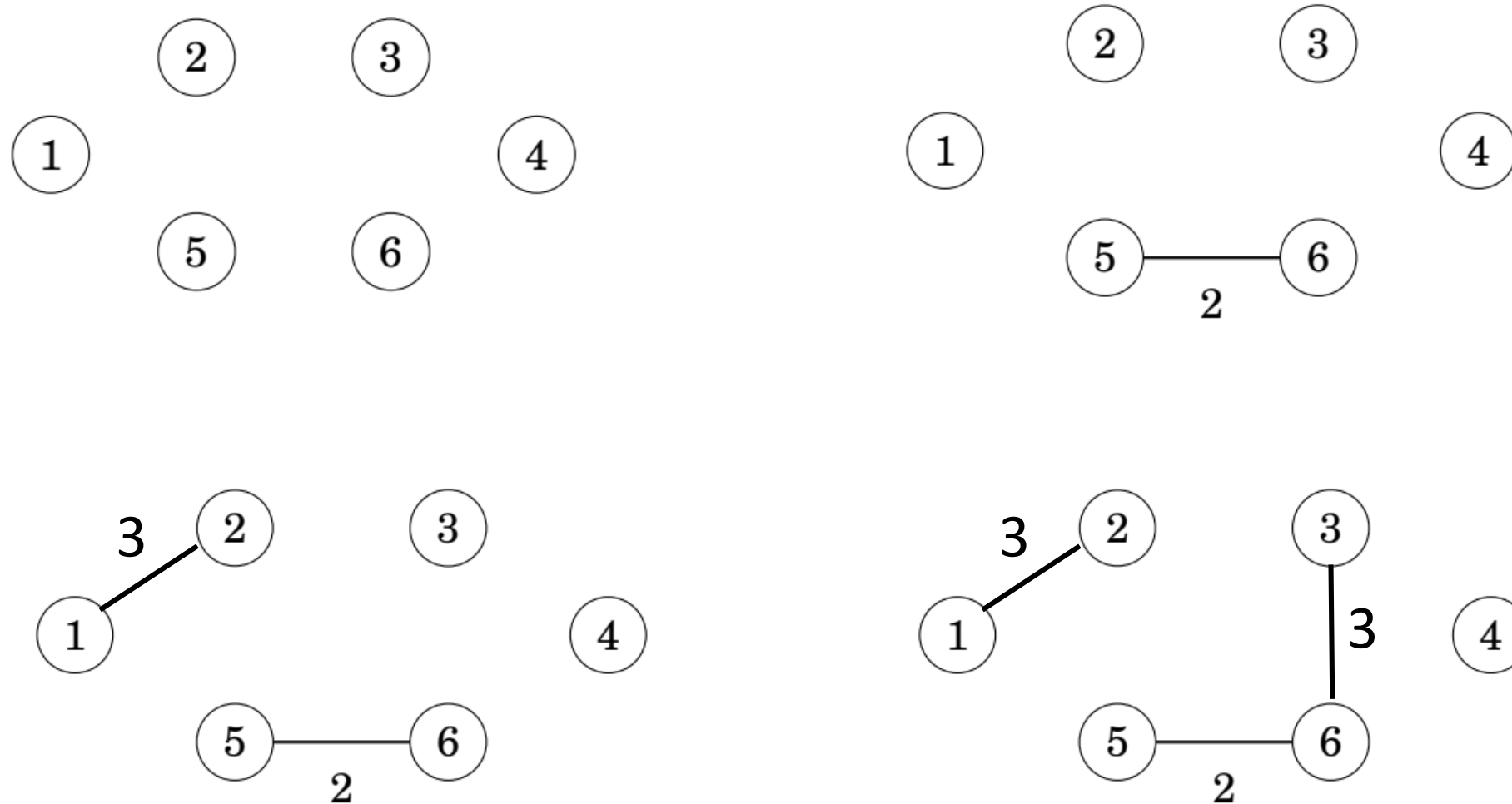
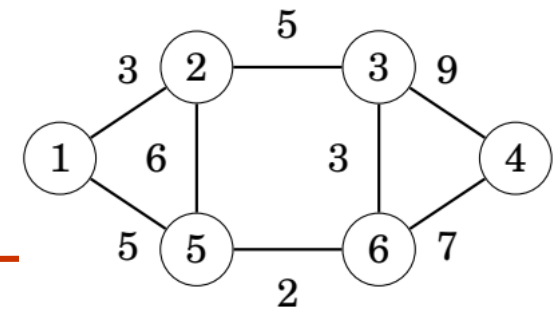
      if (inMST[v] == false && key[v] > weight)
      { key[v] = weight;
        pq.push(make_pair(key[v], v));    // push v into pq.
        parent[v] = u;
      }
    }
  }
}
```

Kruskal's algorithm

- เริ่มต้นจากการเรียงลำดับเอดจ์ (edge) ตามค่าน้ำหนักน้อยไปมาก
- เพิ่มเอดจ์เข้าไปใน MST หากไม่ทำให้เกิด วงวน (cycle) จนกระทั่งเวอร์เท็กซ์ทั้งหมดถูกเลือก

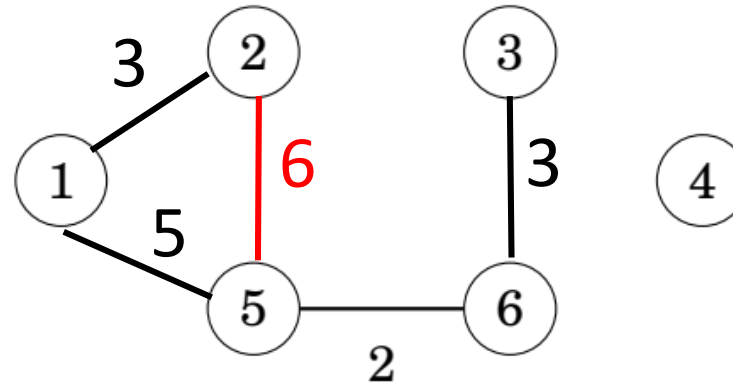
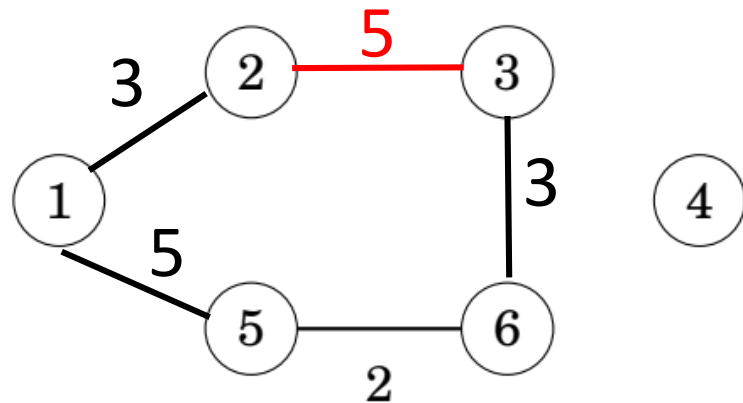
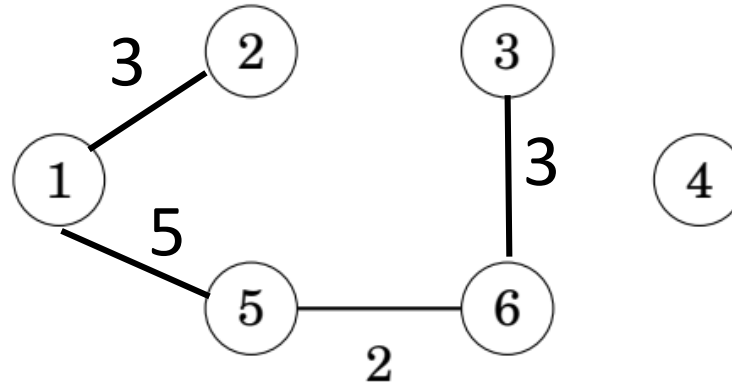
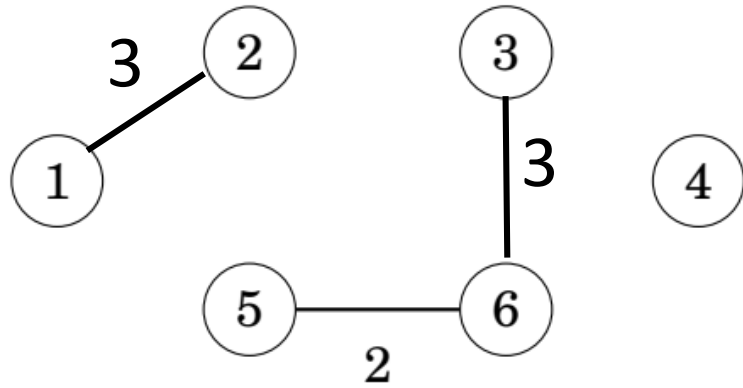
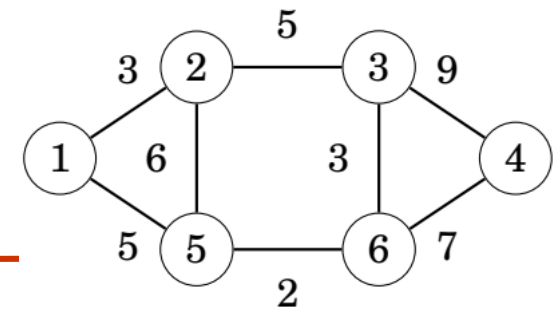


Kruskal's algorithm



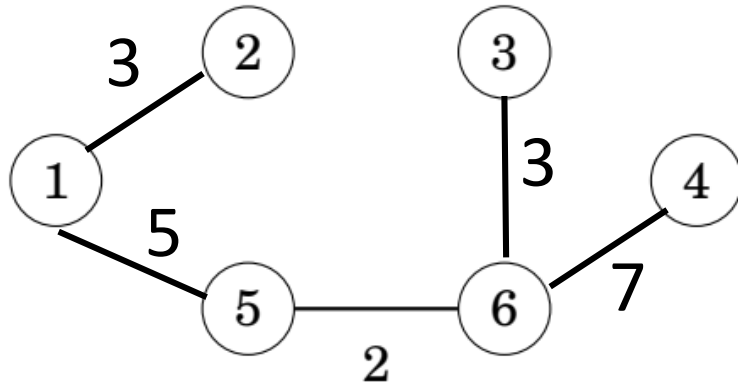
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9

Kruskal's algorithm



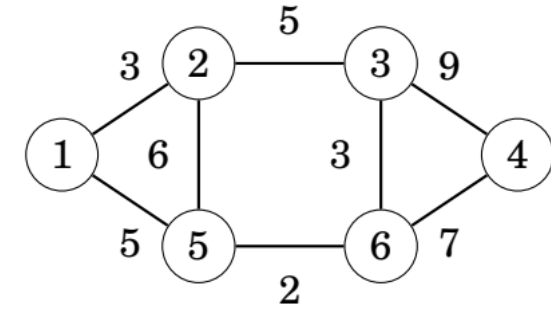
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9

Kruskal's algorithm



MST

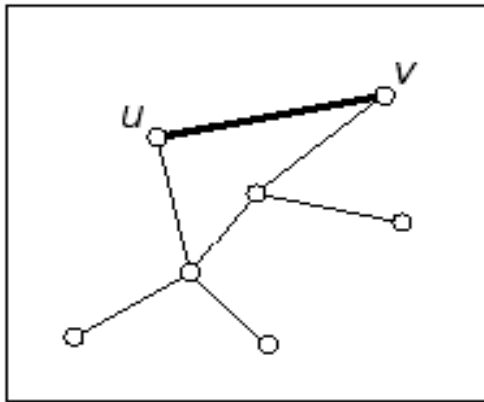
$$3+5+2+3+7 = 20$$



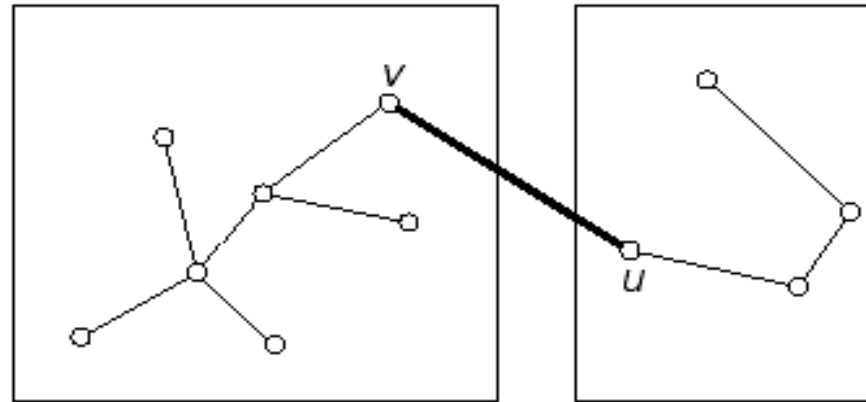
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9

การตรวจสอบวงวน

- ทุกครั้งที่เพิ่มเอดจ์ e_{uv} ต้องเช็คว่ามีเวอร์เท็กซ์ u และ v อยู่ในต้นไม้ย่อยเดียวกันหรือไม่



(a)

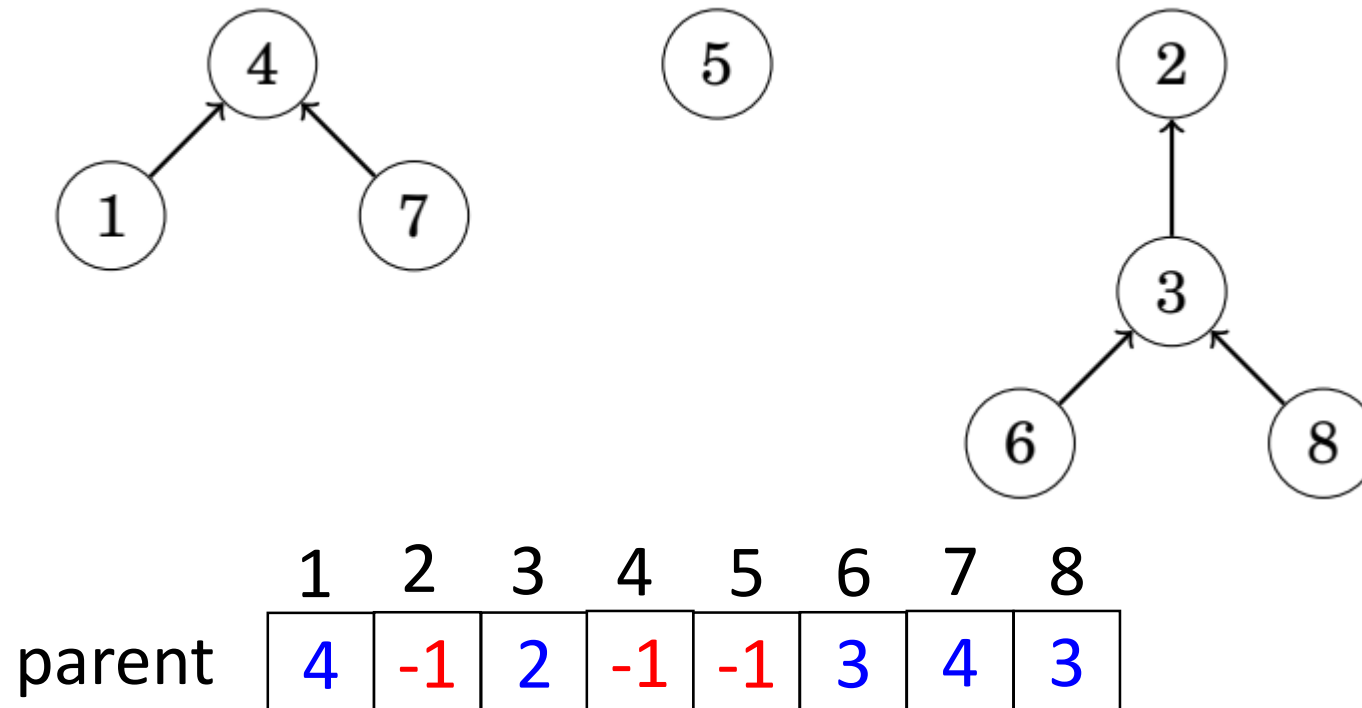


(b)

- ต้องการอัลกอริทึมที่มีประสิทธิภาพที่ใช้ในการตรวจสอบว่าเวอร์เท็กซ์ใดบ้างที่อยู่ในต้นไม้ย่อยเดียวกันบ้าง ซึ่งก็คือ Union-Find structure

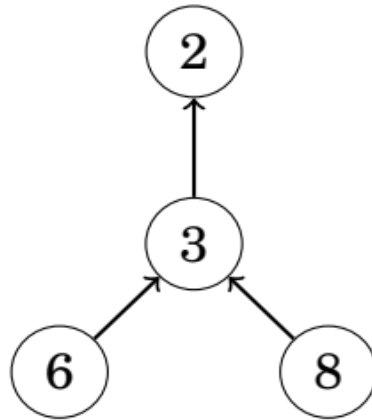
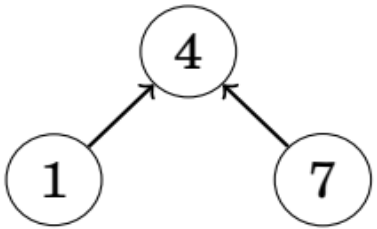
Union-find structure

- Union-find structure เป็นโครงสร้างข้อมูลที่ถูกออกแบบมาเพื่อเก็บ disjoint sets เช่น $\{1,4,7\}$, $\{5\}$, $\{2, 3, 6, 8\}$
- ใน union-find structure จะมีสมาชิก 1 ตัวทำหน้าที่เป็นตัวแทนของเซต



การค้นหาสมาชิกในเซตย่อยเดียวกัน

- หากต้องการตรวจสอบว่า 1 และ 7 อยู่ในเซตเดียวกันหรือไม่ ให้ตรวจสอบว่า 1 และ 7 มีสมาชิกตัวแทนร่วมกันหรือไม่



```

int find(int parent[], int i)
{
    if(parent[i] == -1)
        return i;
    return find(parent, parent[i]);
}
  
```

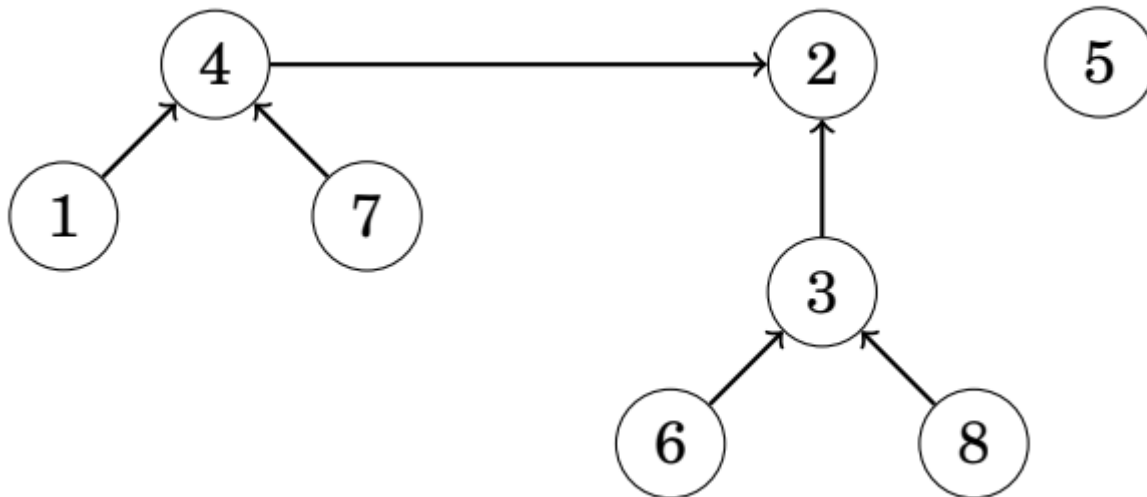
```

int is_cycle(int u, int v)
{
    x = find(parent, u);
    y = find(parent, v);
    return (x==y)? 1:0;
}
  
```

	1	2	3	4	5	6	7	8
parent	4	-1	2	-1	-1	3	4	3

การรวมเซตย่อย

- หากตรวจสอบแล้วไม่พบวงวน ก็จะมี merge เซตย่อยเข้าด้วยกัน เช่น merge {1, 4, 7} เข้ากับ { 2, 3, 6, 8 } จะกลายเป็น {1, 4, 7, 2, 3, 6, 8} โดยเปลี่ยนสถานะของตัวแทนตัวใดตัวหนึ่งในอาร์เรย์ parent



```
void union(int parent[], int i, int j)
{
    parent[j] = i;
}
```

	1	2	3	4	5	6	7	8
parent	4	-1	2	2	-1	3	4	3

Kruskal's algorithm

```
void kruskal(int g[][V])
{ int parent[V], ne=0, a, b, u, v, i, j;
  for(i=0; i<V; i++) parent[i] = -1;

  while(ne < V)
  { for (i = 0, min = inf; i < V; i++)
    for (j = 1; j<V; j++)
      if (g[i][j] < min)
      { min = g[i][j];
        a = u = i;
        b = v = j;
      }

    u = find(parent, u);
    v = find(parent, v);
    if (u != v)
    { ne++;
      union(parent, u, v);
    }
    g[a][b] = g[b][a] = inf;
  }
}
```

```
int find(int parent[], int i)
{ if(parent[i] == -1)
  return i;
  return find(parent, parent[i]);
}
```

```
void union(int parent[],int i, int j)
{ parent[j] = i;
}
```